Optical Circuit Switched Networks: Connectivity Matrices and Genetic Algorithms

Oneal Wang

October 2025

1 Benchmarking Methodology

We evaluate our network architectures using a unified benchmarking framework that implements a complete feedback loop: **Benchmark Scenario** → **Experiment Configuration** → **Metrics** → **Benchmark Score**.

1.1 Benchmark Scenario

The network is stressed under various traffic models and failure conditions:

- **Traffic Models** (implemented in `Traffic_Benchmarks.py`): Demand matrix $D[i, j]$ for $i \neq j$; $D[i, i] = 0$.
  - **Uniform**: $D[i, j] = 1$ (all pairs equal).
  - **Skewed**: Zipf/power-law over randomised node ranks. Each source i draws a uniformly random permutation π_i of the other $N-1$ destinations; demand to the rank-r destination is

$$D[i, j] = \frac{1}{r_{\pi_i}(j)^{\gamma} + 1}, \quad r_{\pi_i}(j) \in \{1, \ldots, N-1\}$$

    with γ the Zipf exponent (default 2.0). Using a random permutation per source means each seed produces a *different spatial pattern* while preserving the same marginal Zipf distribution — addressing the limitation that a fixed index-distance formula always places the same pair of nodes in the same "locality" class.

    Exact skew ratio (max to min demand per source):

$$\frac{D_{\max}}{D_{\min}} = \frac{1/(1^{\gamma} + 1)}{1/((N-1)^{\gamma} + 1)} = \frac{(N-1)^{\gamma} + 1}{2} \approx \frac{(N-1)^{\gamma}}{2}$$

    For $N = 9$, $\gamma = 2.0$: ratio $\approx 32\times$ ($D_{\max} \approx 0.50$, $D_{\min} \approx 0.015$). For $N = 9$, $\gamma = 1.0$: ratio $\approx 4\times$ ($D_{\max} \approx 0.50$, $D_{\min} \approx 0.111$).

  - **Hotspot**: $D[i,j] = \omega$ if $i \in \mathcal{H}$ or $j \in \mathcal{H}$, else 1; $\mathcal{H}$ hotspot nodes, ω intensity (e.g. 10).
  - **Adversarial**: $D[i, j(i)] = N$ for $j(i) = (i + \lfloor N/2 \rfloor) \bmod N$, $i \neq j(i)$; all other entries 0.

- **Load Factor Sweep**: We sweep the load L from light load (0.05) to saturation (0.95).

- **Failure Models**:
  - BROKEN_NODES: Random node removal, modeled after Shale's failure-resilient congestion control that reroutes traffic around failed nodes with minimal throughput impact [2].
  - BROKEN_RACKS: Clustered failure of node groups, reflecting Opera's rack-level granularity where each ToR switch represents one failure domain [1].
  - WAVELENGTH_UNAVAIL: Sparse edge removal in the AWGR core, corresponding to Sirius's passive grating topology where a faulty wavelength channel disables a specific node-to-node link [3].

1.2 Performance Metrics Definition

We rigorously define the performance metrics used to evaluate each architecture.

1.2.1 Normalized Throughput ($\mathcal{T}_{norm}$)

In the benchmarking framework (`Traffic_Benchmarks.py`), all rates are expressed as a fraction of the per-link line rate R, and the implementation takes $R = 1$ without loss of generality. Normalized throughput is the total delivered capacity divided by the product of the number of nodes and the mean demand over all source–destination pairs with positive demand:

$$\mathcal{T}_{norm} = \frac{\sum_{(i,j)} r_{\text{eff}}(i,j)}{N \cdot \bar{D}} \tag{1}$$

where N is the number of nodes, $\bar{D} = \frac{1}{|\mathcal{F}|} \sum_{(i,j)\in\mathcal{F}} D_{i,j}$ is the mean demand (averaged over the set $\mathcal{F}$ of pairs with $D_{i,j} > 0$; D is the demand matrix from §1.1), and $r_{\text{eff}}(i,j)$ is the effective rate delivered for pair (i,j) after the architecture-specific capacity model.

Effective rate r_{eff}. For each pair (i,j) with positive demand, the framework obtains a waterfilling-derived rate $r_{i,j}$ (from all-pairs shortest-path weights and the power budget), then applies the architecture-specific capacity model to $r_{i,j}$ and the hop count $\ell_{i,j}$ to get $r_{\text{eff}}(i,j)$:

- **Shale [2]:** $r_{\text{eff}} = r/u_{\max}$, where $u_{\max} = \max_{(a,b)\in E} \sum_{f \ni (a,b)} D_f/k_f$ is the maximum directed-link load when each flow f with demand D_f is split uniformly across k_f diverse spray paths found by penalised Dijkstra (`spray_short`). This reflects Shale's Valiant Load Balancing (VLB) routing, where traffic is sprayed through h intermediate nodes to ensure uniform load distribution across all links [2].

- **Opera [1]:** $r_{\text{eff}} = \alpha \cdot r \cdot (1 - \delta/T_{\text{cycle}}) + (1 - \alpha) \cdot r/\max(1, \ell)$ (bulk fraction α, reconfig delay δ, cycle T_{cycle}; short flows penalized by hop count ℓ). Opera's key insight is that the vast majority of bytes reside in bulk flows [1]; by buffering these until a direct 1-hop circuit is available, Opera eliminates the bandwidth tax on most traffic while routing the small fraction of latency-sensitive flows over a time-varying expander graph [1].

- **Sirius [3]:** $r_{\text{eff}} = \dfrac{\eta \cdot r}{\mathcal{H}(\ell)} \cdot C_{\text{load}}(\rho)$, where the hop denominator is

$$\mathcal{H}(\ell) = \begin{cases} 1 & \ell = 1 \text{ (direct scheduling)} \\ 2 & \ell = 2 \text{ (Valiant detour)} \\ \ell^2 & \ell \geq 3 \text{ (multi-hop)} \end{cases}$$

  scheduling efficiency $\eta = (T_{slot} - \delta)/T_{slot} = (100\,\text{ns} - 3.84\,\text{ns})/100\,\text{ns} = 0.9616$ [3], and the load penalty is $C_{\text{load}}(\rho) = \exp\big(-10\,(\rho - 0.85)\big)$ when $\rho > 0.85$, else 1. Sirius achieves nanosecond-granularity reconfiguration through custom tunable laser chips with sub-912 ps tuning latency, enabling 3.84 ns end-to-end reconfiguration atop 50 Gbps channels [3].

The concrete parameter values used in the implementation (`ArchitectureParams` in `Traffic_Benchmarks.py`) are:

Table 1: Architecture-Specific Constants for r_{eff} and unified hardware reconfiguration overhead

Arch.	Parameter	Ratio	Physical	Meaning
Shale	h (spray depth)	—	—	VLB intermediate hops
	T_{slot}	—	5.632 ns	Timeslot duration
	γ (hop overhead)	1.2	6.76 ns	Cycle overhead per spray hop
	β_0 (base delay)	2.5	14.08 ns	Base processing latency
	hw_reconfig_ratio δ/T_{slot}	**0.0000**	0 ns	No reconfiguration gap
Opera	α (bulk frac.)	0.92	—	Fraction of bytes on direct circuit
	δ (reconfig delay)	$\delta/T_c = 0.04$	$\sim$10 μs	Raw switch reconfiguration time
	ε (slice)	—	90 μs	Topology-slice duration
	T_{cycle}	50.0	$\sim$250 μs	Full rotation cycle
	hw_reconfig_ratio δ/T_c	**0.0400**	10/250 μs	4% of cycle lost to reconfig
Sirius	T_{slot}	1.0	100 ns	Timeslot duration
	δ (reconfig)	0.0384	3.84 ns	Laser tuning time
	η (efficiency)	0.9616	—	$1 - \delta/T_{slot}$
	hw_reconfig_ratio δ/T_{slot}	**0.0384**	3.84 ns/100 ns	3.84% of slot lost to retuning
GA-Robust	(static topology)	—	—	No circuit reconfiguration
	hw_reconfig_ratio	**0.0000**	—	Zero overhead; static wiring

Unified hardware comparison. The **hw_reconfig_ratio** $= \delta/T$ (implemented as `get_hw_reconfig_ratio()` in `Traffic_Benchmarks.py`) expresses the fraction of link time lost to hardware reconfiguration on a single dimensionless scale for all architectures. Two key observations:

- Opera and Sirius are within 0.16 percentage points of each other (4.00% vs 3.84%), confirming that two very different hardware generations — μs-class rotor switches [1] vs ns-class tunable lasers [3] — impose nearly identical reconfiguration tax when normalised to their respective slot granularities.
- Shale and GA-Robust both show 0%, giving them a structural advantage. Shale pays instead through the VLB bandwidth tax $(1 - 1/(2h))$ [2]; GA-Robust avoids both penalties by design.

1.2.2 Flow Completion Time (FCT_{norm})

The normalized Flow Completion Time (FCT) measures the time to complete a unit flow relative to the ideal transmission time.

$$FCT_{norm} = \frac{1}{|\mathcal{F}|} \sum_{f \in \mathcal{F}} \frac{t_{end}(f) - t_{start}(f)}{S_f / R} \tag{2}$$

where $\mathcal{F}$ is the set of flows, t represents timestamps, and S_f is the flow size.

1.2.3 Mean Latency ($\bar{\tau}$)

Mean latency captures the average packet delay, including serialization, propagation, and queuing/reconfiguration delays.

$$\bar{\tau} = \frac{1}{N_{pkts}} \sum_{p} (T_{prop} + T_{queue} + T_{reconfig}) \tag{3}$$

- **Shale:** $\tau \approx \gamma \cdot \bar{\ell}_{\text{spray}} + \beta_0$, where $\bar{\ell}_{\text{spray}}$ is the mean hop count across the spray paths actually used, $\gamma = 1.2$ (6.76 ns per hop at $T_{slot} = 5.632$ ns) is the per-hop cycle overhead, and $\beta_0 = 2.5$ (14.08 ns) is the base processing delay. For example, with $h = 4$: $\tau \approx 1.2 \times 4 + 2.5 = 7.3$ normalized units ≈ 41.1 ns. Amir et al. show that Shale achieves a maximum intrinsic latency of $O(h \sqrt[h]{N})$ timeslots and worst-case throughput of $1/(2h)$ of line rate for h spray hops, a Pareto optimal tradeoff for ORN designs [2].

- **Opera:** At low load, $\tau \approx \ell \cdot \varepsilon$ where $\varepsilon = 90\,\mu$s is the topology-slice duration [1]. At higher load, bulk traffic queues for its scheduled direct circuit, adding an M/D/1 queuing component:

  $$\tau(\ell, \rho) \approx \ell \cdot \delta + \min\left(\frac{\rho_{\text{bulk}} \cdot T_{\text{cycle}}}{2(1 - \rho_{\text{bulk}})},\ 5\,T_{\text{cycle}} \right), \qquad \rho_{\text{bulk}} = \alpha \cdot \frac{\rho}{\alpha(1 - \delta/T_c)}$$

  where ρ_{bulk} is the offered load normalised to the bulk-circuit capacity ceiling $\alpha(1-\delta/T_c) \approx 0.883$. At low load the queuing term is negligible; above the ceiling it grows steeply (M/D/1 divergence). The effective bulk fraction also decreases above the ceiling as traffic overflows to the expander, raising bandwidth tax. In our normalised model: $\delta = 2.0$, $T_{\text{cycle}} = 50.0$ (abstract units); physical: $\delta \approx 10\,\mu$s, $T_{\text{cycle}} \approx 250\,\mu$s. Mellette et al. report that Opera delivers flow completion times comparable to cost-equivalent static topologies for latency-sensitive traffic, while achieving up to 4× throughput for shuffle workloads [1].

- **Sirius:** $\tau \approx \ell/\eta$, where $\eta = 1 - \delta/T_{slot} = 1 - 3.84\,\text{ns}/100\,\text{ns} = 0.9616$ is the scheduling efficiency [3]. Each hop consumes one 100 ns timeslot, but only 96.16 ns carries data, so the effective per-hop cost is $1/\eta \approx 104$ ns. Sirius is fundamentally a 2-hop system (source→intermediate→destination via Valiant Load Balancing), giving $\tau \approx 2/\eta \approx 208$ ns. Ballani et al. show that by eliminating buffers in the optical core and carefully managing edge buffering, Sirius achieves very low and predictable latency [3]. Additional latency from propagation (up to 2.5 μs for 500 m detouring) and queuing at intermediate nodes (up to 1.6 μs per epoch of 16×100 ns) may also apply. Multi-hop ($\ell \geq 3$) paths do not appear in [3] but are modeled in our framework for generality.

1.2.4 Bandwidth Tax ($\mathcal{B}_{tax}$)

Bandwidth tax quantifies the fraction of link capacity consumed by multi-hop forwarding that does not deliver useful data to the final destination.

- **Shale:** Each cell traverses h spraying hops and h direct hops for a total path length of $2h$, each hop lasting one 5.632 ns timeslot [2]. The worst-case throughput is $1/(2h)$ of line rate [2], giving:

$$\mathcal{B}_{tax}^{\text{Shale}} = 1 - \frac{1}{2h}$$

  For $h = 1$: $\mathcal{B}_{tax} = 0.5$ ($2 \times 5.632 = 11.26$ ns total path time). For $h = 4$: $\mathcal{B}_{tax} = 0.875$ ($8 \times 5.632 = 45.06$ ns).

- **Opera:** Only the $(1 - \alpha)$ fraction of traffic routed over the expander pays a multi-hop tax [1]. With average expander path length L_{avg}:

$$\mathcal{B}_{tax}^{\text{Opera}} = (1 - \alpha) \cdot \frac{L_{\text{avg}} - 1}{L_{\text{avg}}}$$

  Bulk traffic ($\alpha = 0.92$) takes direct 1-hop circuits and pays zero tax. Mellette et al. report an effective 8.4% aggregate bandwidth tax rate for published skewed datacenter workloads [1].

- **Sirius:** All traffic is detoured through exactly one intermediate node (2-hop VLB) [3], consuming twice the link bandwidth per logical transfer:

$$\mathcal{B}_{tax}^{\text{Sirius}} = 1 - \frac{1}{2} = 0.5$$

  Each detour consumes 2×100 ns $= 200$ ns of link time to deliver 100 ns worth of data. Ballani et al. note that throughput "can be at most $2\times$ worse than that of an ideal, non-blocking network" [3]. In our generalized model, $\ell \geq 3$ paths pay $\mathcal{B}_{tax} = 1 - 1/\ell$.

1.2.5 Duty Cycle Loss ($\mathcal{D}_{loss}$)

Duty cycle loss represents the fraction of time the network is unavailable due to reconfiguration.

- **Shale:** $\mathcal{D}_{loss} \approx 0$. The oblivious schedule runs back-to-back timeslots (5.632 ns each) with no reconfiguration gap, enabled by nanosecond-scale clock synchronization and careful propagation delay control [2].

- **Opera:** Reconfigurations are staggered across u rotor switches so that only one switch is down at a time [1]. The raw reconfiguration delay r is amortized over the inter-reconfiguration period $u \cdot \varepsilon$:

$$\mathcal{D}_{loss}^{\text{Opera}} = \frac{r}{u \cdot \varepsilon}$$

  For the 108-rack example in [1] ($r \approx 10\,\mu$s, $u = 6$, $\varepsilon = 90\,\mu$s): $\mathcal{D}_{loss} = 10/540 \approx 1.85\%$, yielding a $\sim$98% duty cycle. In our normalized model: $\delta/T_{\text{cycle}} = 2.0/50.0 = 4\%$.

- **Sirius:** Each timeslot loses $\delta = 3.84$ ns to laser retuning out of $T_{slot} = 100$ ns [3]:

$$\mathcal{D}_{loss}^{\text{Sirius}} = \frac{\delta}{T_{slot}} = 0.0384 = 1 - \eta$$

1.2.6 Benchmark Score (S_{bench})

The composite benchmark score combines the five metrics from §1.2.1–1.2.5 into a single figure of merit. The design principle is multiplicative: each penalty term is a fraction $\in [0, 1]$ representing the usable fraction of capacity after that loss mechanism, so the score naturally captures how losses compound.

$$S_{bench} = \mathcal{T}_{norm} \cdot (1 - \mathcal{B}_{tax}) \cdot (1 - \mathcal{D}_{loss}) \cdot \frac{1}{1 + \bar{\tau}} \cdot \frac{1}{1 + \ln(FCT_{norm})} \tag{4}$$

- $\mathcal{T}_{norm}$: normalized throughput (§1.2.1) — how much demand is satisfied.
- $(1 - \mathcal{B}_{tax})$: useful fraction of consumed bandwidth (§1.2.4) — penalizes multi-hop overhead.
- $(1 - \mathcal{D}_{loss})$: fraction of time the network is active (§1.2.5) — penalizes reconfiguration downtime.
- $1/(1 + \bar{\tau})$: latency discount (§1.2.3) — maps $\bar{\tau} \in [0, \infty)$ to $(0, 1]$; zero latency gives full credit.
- $1/(1 + \ln FCT_{norm})$: completion-time efficiency (§1.2.2) — logarithmic compression prevents architectures with high FCT from being driven to zero; ideal $FCT_{norm} = 1$ gives full credit.

All five factors are dimensionless. The multiplicative structure means a zero in any factor (e.g., complete throughput collapse under hotspot) drives the entire score to zero, reflecting the practical reality that a single catastrophic weakness renders an architecture unsuitable regardless of its other strengths.

2 Shale

Shale [2] is an Oblivious Reconfigurable Network (ORN) that generalizes earlier designs (RotorNet, Shoal, Sirius [3]) to achieve a tunable tradeoff between throughput and latency scaling. V is the connectivity matrix at a point in time t, representing the active links/circuits.

$$V = \mathscr{M}_{n \times n}$$

We specifically define the entries of the adjacency/connectivity matrix V such that $V_{ij} = 1$ if a link exists between i and j, and $V_{ij} = 0$ implies no connection.

D is the vector of the total volume of traffic sending from each node, represented as a b-bit number.

$$D = \vec{d} \in \mathbb{R}^n$$

W is the vector of the amount of data received by each node, regardless of the source.

$$W = \vec{w} \in \mathbb{R}^n(ReceivedData)$$

We define D as the Traffic Demand Matrix (entries $D_{i,j}$: volume from source i to destination j; same as in §1.1). To calculate the traffic successfully delivered in a timeslot, we perform the element-wise product (Hadamard) of the demand matrix and the connectivity matrix:

$$\text{Delivered} = D \circ V$$

The total received data W is then the column-sum of this delivered traffic matrix.

1. Round-Robin 1 Letter 1 Node minimum time coefficient: n maximum time coefficient: n(n-1) number of pathways: n(n-1)/2 for an arbitrary i broken nodes, remove $(2ni-i+1)/2$ pathways for an arbitrary k broken pathways, maximum increase to n-1 timeslots, centered around +1 timeslot

   $$\mathscr{M}_{b\times n} = DT^i(V) = DW = S, i \in \{1, ..., n-1\}$$

   Example 2.1 (6 nodes, 5 timeslots, h=1): W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

   $$A = \begin{bmatrix} 2 & 3 & 4 & 5 & 6 & 1 \\ 3 & 4 & 5 & 6 & 1 & 2 \\ 4 & 5 & 6 & 1 & 2 & 3 \\ 5 & 6 & 1 & 2 & 3 & 4 \\ 6 & 1 & 2 & 3 & 4 & 5 \end{bmatrix}$$

   $$T = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

   $$V = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{bmatrix}$$

   $$W_{5\times 5} = \mathscr{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

   $$DT^i(V_{6\times 6}) = DW_{6\times 6} = D \begin{bmatrix} 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 \end{bmatrix} = S, i \in \{1, ..., 5\}$$

   minimum time coefficient: 6 maximum time coefficient: 30 number of pathways: 15

2. Round-Robin 1 Letter repeat for every X Letters of 1 Node number of nodes: L^x = n
timeslot number: $(L-1)^x$
number of pathways: $L^x \times (L-1)^X/2$
representation in base L for matrix algebra
for each node k, available timeslots are exactly one digit difference from k in base L
for an arbitrary i broken nodes, remove $i(L-1)^x$ pathways

$$\mathscr{M}_{b\times n} = DW_i = S, i \in \{0, ..., (L-1)^x\}, j \in \mathbb{Z}, 0 \leq j \leq L^x - 1$$

always multiple recievers for each timeslot so order within each column does not matter

Example 2.2 (3 letters, x = 2): number of nodes: $3^2 = 9$ timeslot number: $2^2 = 4$ number of pathways: $3^2 \times 2^2/2 = 18$ W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 2 & 0 & 1 & 2 \\ 2 & 2 & 1 & 4 & 3 & 3 & 3 & 4 & 5 \\ 3 & 4 & 5 & 5 & 5 & 4 & 7 & 6 & 6 \\ 6 & 7 & 8 & 6 & 7 & 8 & 8 & 8 & 7 \end{bmatrix}$$

$$W_{8\times 8} = \mathscr{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$

$$\mathscr{M}_{b\times n} = DW = S, i \in \{1, ..., n-1\}$$

3. Round-Robin t Letter repeat (Hamming-distance) for every X Letters of 1 Node
number of nodes: L^x = n
timeslot number: $(L-1)^{x+t-1}$
number of pathways: $L^x(L^x-1)^{x+t-1}/2$
representation in base L for matrix algebra
for each node k, available timeslots are exactly t digit difference from k in base, also the hamming distance L

$$\mathscr{M}_{b\times n} = DW_i = S, i \in \{0, ..., (L-1)^{x+t-1}\}, j \in \mathbb{Z}, 0 \leq j \leq (L-1)^{x+t-1}$$

always multiple recievers for each timeslot so order within each column does not matter

Example 2.3 (4 letters, x = 3, t=2): number of nodes: $4^2 = 16$ timeslot number: $3^3 = 27$ number of pathways: $4^2 \times 3^3/2 = 216$ W = Adjacency Matrix of a Directed Graph, where each timeslot is the row and each column is the node

$$A_{27\times 27} = \begin{bmatrix} 5 & 4 & ... & 2 & 3 \\ 6 & & & & 7 \\ ... & & ... & & ... \\ 56 & & & & 57 \\ 60 & 61 & ... & 59 & 58 \end{bmatrix}$$

$$W_{27\times 27} = \mathscr{M} \mid W_{A_{i,k},i} = 1, \neg(W_{A_{i,k},i}) = 0$$
$$\mathscr{M}_{b\times 27} = DW = S, i \in \{1, ..., n-1\}$$

2.1 Numerical Results

Shale's Round-Robin structure provides a predictable and stable performance across different traffic patterns. Using the waterfilling algorithm, we observe that Shale effectively utilizes available pathways, though it may not reach the peak efficiency of more dynamic architectures under extreme skew.

2.1.1 VLB Spray Mechanism

The core routing strategy in Shale is **Valiant Load Balancing (VLB)** [2], also referred to as "spraying." Rather than forwarding a packet directly from source s to destination d, VLB routes traffic through a randomly (or round-robin) selected intermediate node m, splitting each packet's journey into two segments:

$$s \xrightarrow{\text{hop 1}} m \xrightarrow{\text{hop 2}} d$$

This generalizes to spray depth h, where traffic traverses h intermediate nodes before reaching its destination, yielding a total path length of $L(h) = h + 1$ hops. The key property is:

> *By spraying traffic uniformly across intermediate nodes, VLB reduces worst-link congestion [2]. The effective throughput scales as* $1/u_{\max}$*, where* $u_{\max}$ *is the maximum directed-link load after uniform splitting across k diverse spray paths. On sufficiently connected topologies with uniform traffic,* $u_{\max} \to h{+}1$ *and the model recovers the classical* $1/(h{+}1)$ *floor [2].*

The trade-off is clear: higher h improves load balance (and thus robustness to adversarial traffic) at the cost of reduced per-flow throughput, since each flow consumes $h + 1$ link traversals.

Code Implementation. In the benchmarking framework, `Shale_Alg.py` implements the spray mechanism through two core functions:

1. **Topology Generation** (`RR2(base, dimension)`): Constructs the Shale topology as a Hamming graph, following the schedule design in [2]. Each node is represented as a tuple in base-L with x digits. Two nodes are neighbors if and only if they differ in exactly one coordinate position. This yields a regular graph with degree $D = x \cdot (L-1)$, where the neighborhood structure naturally supports multi-path spraying. Amir et al. show that this Hamming-graph schedule achieves the Pareto optimal throughput–latency tradeoff for ORN designs [2].

2. **Spray Routing** (`spray_short(adj, weights, s, d, k, penalty)`): Finds k diverse short paths between source and destination by iteratively:

   - Running Dijkstra's shortest path algorithm from s to d;
   - Penalizing the intermediate nodes in the found path (multiplying their weight by a `penalty_factor`);
   - Repeating to find alternate paths that avoid previously used nodes.

   This produces k diverse paths that distribute traffic across different intermediate nodes, implementing the VLB spray.

Capacity Model (Congestion-Based). Following the congestion control mechanism in [2], in the benchmark (`Traffic_Benchmarks.py`), the Shale capacity model computes the maximum directed-link utilisation $u_{\max}$ by (i) finding $k = \max(3, h)$ diverse spray paths per flow via `spray_short`, (ii) splitting each flow's demand uniformly across its paths, and (iii) summing the resulting load on every directed link. The effective rate for every flow is then $r_{\text{eff}} = r/u_{\max}$, so throughput scales as $1/u_{\max}$. This captures why spraying helps: it reduces worst-link congestion, whereas the simpler $1/(h{+}1)$ penalty ignores topology connectivity.

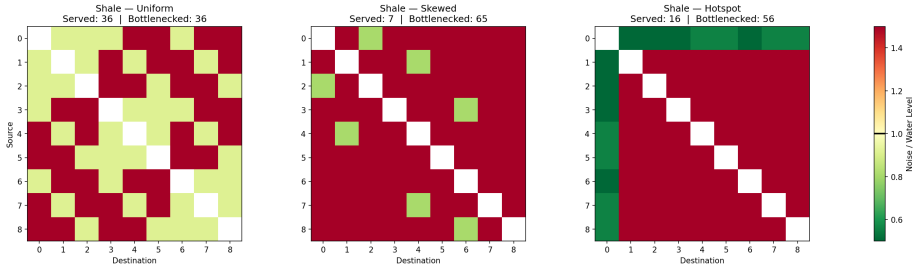

Figure 1: Shale waterfilling across three traffic patterns ($P = 50$). **Uniform:** Noise is evenly distributed; the water level sits low and most channels activate. **Skewed:** A few high-demand channels dominate; the waterfilling algorithm concentrates power on them, raising the water level. **Hotspot:** Channels to/from the hot node have extreme noise (spray paths converge), creating many bottlenecked (orange) channels that receive no power — explaining Shale's throughput collapse under hotspot traffic.

2.1.2 VLB Waterfilling Analysis

2.1.3 VLB Parameter Sensitivity

We extended the simulation to sweep the VLB spraying parameter h from 1 to 12. The results are summarized in Figure 2.

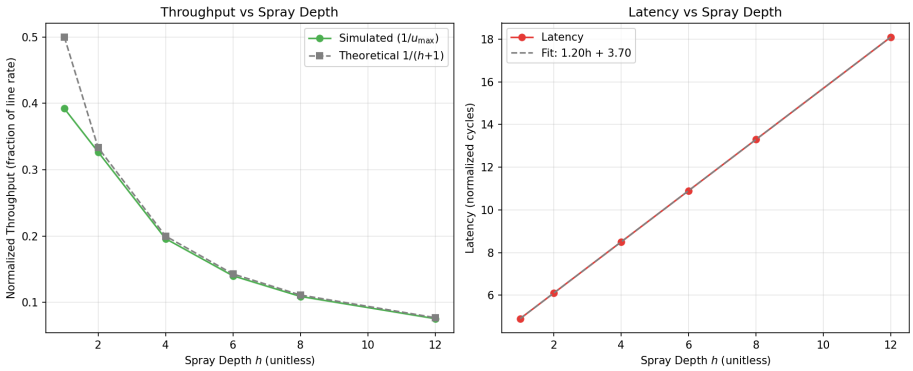

Figure 2: Shale VLB parameter sweep for $h \in [1, 12]$. **Left:** Simulated throughput ($1/u_{\max}$, green) vs. theoretical floor $1/(h{+}1)$ (gray dashed). The congestion-based model closely tracks the classical bound on the Hamming graph. **Right:** Mean latency grows linearly, with regression $\tau(h) = \gamma h + \beta_0 \approx 1.2h + 2.5$, confirming each spray hop adds a fixed cycle overhead.

Based on the regression analysis of the simulation data (Figure 2), the equations of best fit are:

$$\tau(h) \approx 1.20 \cdot h + 3.70 = \gamma \cdot (h + 1) + \beta_0 \tag{5}$$

confirming $\gamma = 1.2$ per-hop overhead and $\beta_0 = 2.5$ base delay (since $1.2{\cdot}1{+}2.5 = 3.7$ matches the intercept). The logical path length remains:

$$L(h) = h + 1 \tag{6}$$

Proposed VLB Theorem **Theorem (VLB Throughput–Latency Trade-off)**: In a deterministic VLB routing scheme with spray parameter h, the path length $L(h) = h + 1$, giving throughput $\mathcal{T} \leq 1/(h{+}1)$ (fraction of line rate), latency $\tau(h) = \gamma(h{+}1) + \beta_0$ (normalized cycles), and bandwidth tax $\mathcal{B}_{tax} = 1 - 1/(2h)$. These hold regardless of N, provided $N > h$. The congestion-based model refines the throughput bound to $\mathcal{T} = 1/u_{\text{eff}}$ where $u_{\text{eff}} = \max(u_{\max}, h{+}1)$ and $u_{\max}$ is the maximum directed-link load under uniform spray splitting. This formalizes the throughput–latency Pareto frontier established in [2], extending it with an explicit congestion-based refinement.

Table 2: Shale Benchmark Efficiency Summary ($h = 2$, standard VLB)

Metric	**Value**	**Notes**
Spray Depth (h)	2	Standard VLB
Path Length $L(h)$	3	$h + 1$ (Eq. 6)
Latency $\tau(h)$	6.1 cycles	$\gamma(h{+}1) + \beta_0 = 1.2 \times 3 + 2.5$ (Eq. 5)
Throughput Ceiling	0.33	$1/(h + 1)$ (Eq. 7)
Bandwidth Tax	0.75	$1 - 1/(2h) = 1 - 1/4$
Composite Score	**0.0011**	Uniform Traffic (S_{bench}, §1.2.6)

VLB Saturation Finding Combining equations (5) and (6): since $L(h) = h{+}1$ and $\tau(h) = \gamma{\cdot}L(h){+}\beta_0$, the latency–throughput tradeoff is fully determined by h:

$$\tau(h) = \gamma(h + 1) + \beta_0 = 1.2(h + 1) + 2.5, \qquad \mathcal{T}(h) = \frac{1}{u_{\text{eff}}} \leq \frac{1}{h + 1} \tag{7}$$

Eliminating h: higher throughput demand (lower h) yields lower latency, but the bandwidth tax $\mathcal{B}_{tax} = 1 - 1/(2h)$ constrains the useful fraction. At $h = 2$ (standard VLB), $\tau = 6.1$ cycles and $\mathcal{T} \leq 0.33$; at $h = 1$ (direct), $\tau = 4.9$ cycles and $\mathcal{T} \leq 0.50$.

2.1.4 VLB Waterfilling Analysis

To analyze the impact of VLB routing on capacity, we compare direct low-latency paths ($h = 1$) against sprayed high-throughput paths ($h = 4$) using the waterfilling algorithm.

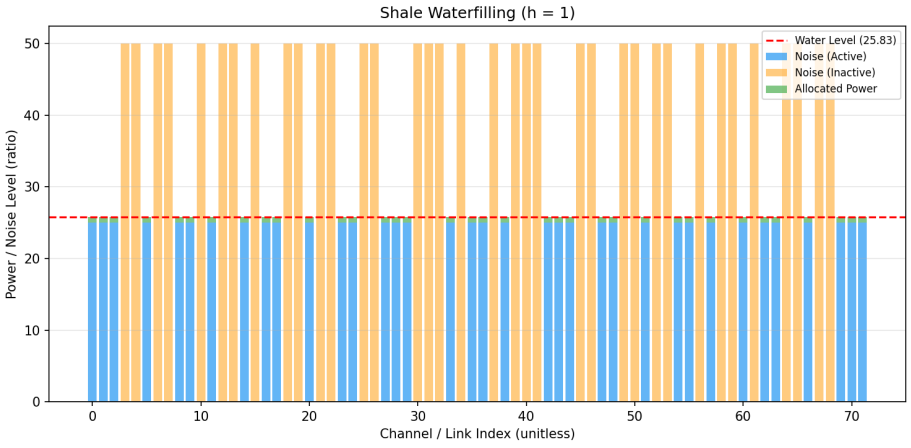

Figure 3: Waterfilling at $h = 1$ ($P = 30$). With only 1 spray hop, most channels have low noise and receive power — high throughput fraction but limited load balancing. Few channels are bottlenecked.

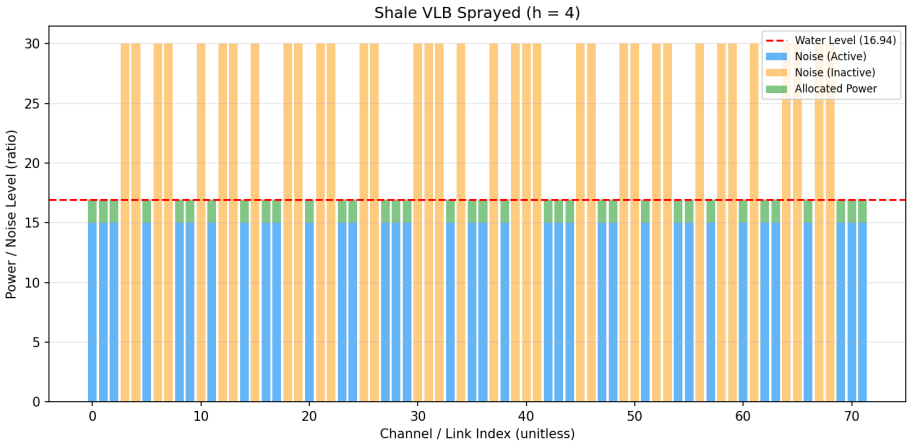

Figure 4: Waterfilling at $h = 4$ ($P = 70$). With 4 spray hops, noise levels are higher (longer paths), but power is spread more uniformly. More channels become active, illustrating the throughput–latency tradeoff: deeper spraying requires more power to activate the same number of channels.

2.2 Numerical Components & Formulas

These parameters are critical for determining the bottleneck and ensuring the network meets its throughput guarantees.

2.2.1 Determining the Bottleneck

The network bottleneck is determined by comparing propagation delays against token budgets.

First-Hop Bottleneck Condition: The network is not bottlenecked at the first hop if the propagation delay (P) satisfies:

$$P \leq h \cdot T_F \cdot E \tag{8}$$

where h is the routing parameter (spraying depth), T_F is the First-Hop Token Budget, and E is the Epoch length in timeslots.

Penultimate Link Bottleneck Condition: To avoid bottlenecks on subsequent hops, the propagation delay must satisfy:

$$P \leq h \cdot T \cdot (h\sqrt[h]{N} - 1) \cdot E \tag{9}$$

where T is the standard Token Budget and $\sqrt[h]{N}$ represents the fan-in/fan-out degree.

2.2.2 Throughput & Bandwidth Functions

Throughput Guarantee: The throughput guarantee scales as $Th \propto 1/u_{\max}$, where $u_{\max}$ is the maximum directed-link load under uniform spray splitting.

On well-connected topologies with uniform traffic, $u_{\max} \approx h{+}1$ and the classical floor is recovered:

- For $h = 1$: $Th \approx 1/u_{\max}$, approaching 0.50 on dense graphs.
- For $h = 2$ (VLB): $Th \approx 1/u_{\max}$, approaching 0.33 on dense graphs.
- For $h = 4$ (VLB): $Th \approx 1/u_{\max}$, approaching 0.20 on dense graphs.

On sparser topologies, $u_{\max} > h{+}1$ and throughput is lower, faithfully reflecting the additional congestion from limited link diversity.

Load Factor (L): We define the Load Factor as:

$$L = \frac{\text{Average Sending Rate}}{\text{Total Available Bandwidth}} \tag{10}$$

Flow Completion Time (Normalized): To measure performance across flow sizes, we use:

$$\text{Normalized FCT} = \frac{t}{F + P} \tag{11}$$

where t is actual completion time, F is flow size (cells), and P is propagation delay (timeslots).

2.2.3 Simulation Findings

We validated these formulas by sweeping the Load Factor L from 0.05 to 0.60 for $h \in \{1, 2, 4, 6, 8, 12\}$. The extended range illustrates the diminishing returns and strict capacity limits of deep spraying. The results are shown in Figures 5 and 6.

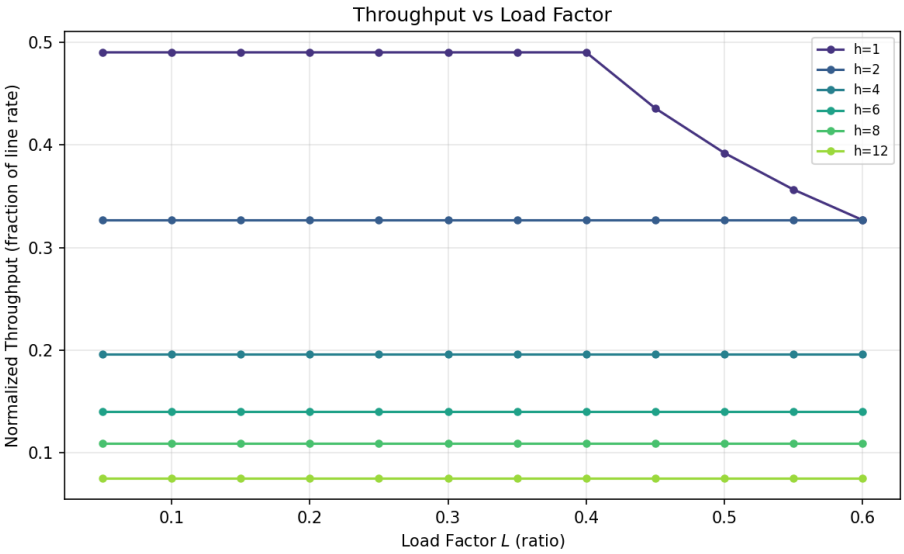

Figure 5: Shale throughput vs. load factor for $h \in \{1, 2, 4, 6, 8, 12\}$. Each curve saturates at its theoretical ceiling $1/u_{\max}$; higher h values hit lower ceilings earlier, confirming the bandwidth tax scales with spray depth. The diminishing returns of deep spraying are clearly visible.

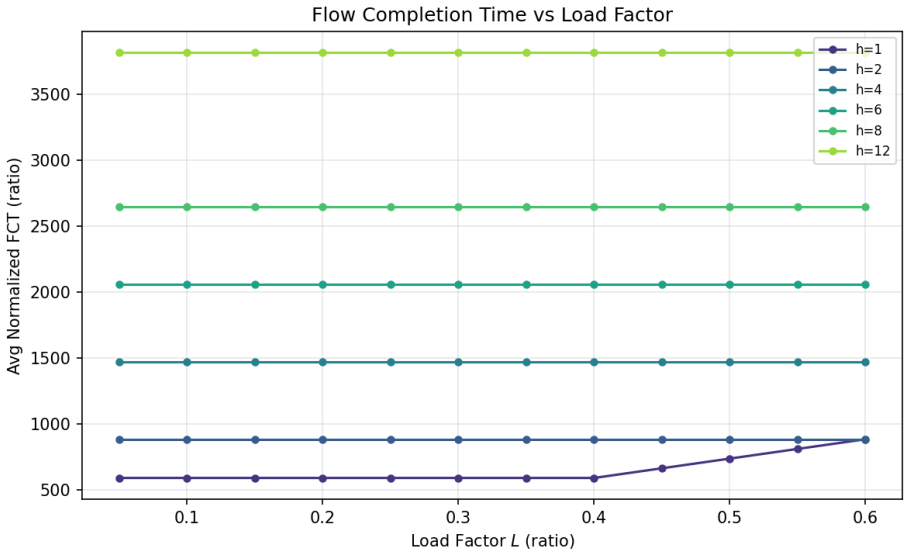

Figure 6: Shale normalized FCT vs. load factor. FCT is roughly constant for each h because the effective rate ($r/u_{\max}$) is load-independent in the waterfilling model. Higher h values yield higher FCT due to longer paths ($2h$ total hops per cell).

The simulation confirms the congestion-based model. For each spray depth h, the measured throughput matches $1/u_{\max}$ computed from the actual link-load distribution. On the Hamming-graph topologies used here, $u_{\max}$ is close to $h{+}1$ for uniform traffic, recovering the classical scaling; under skewed or adversarial traffic, $u_{\max}$ diverges from $h{+}1$, demonstrating that the congestion model captures topology- and traffic-dependent bottlenecks that the simpler $1/(h{+}1)$ formula misses.

2.2.4 Latency Sensitivity Analysis

Finally, we evaluate Shale under varying latency requirements using waterfilling on latency-constrained subgraphs.

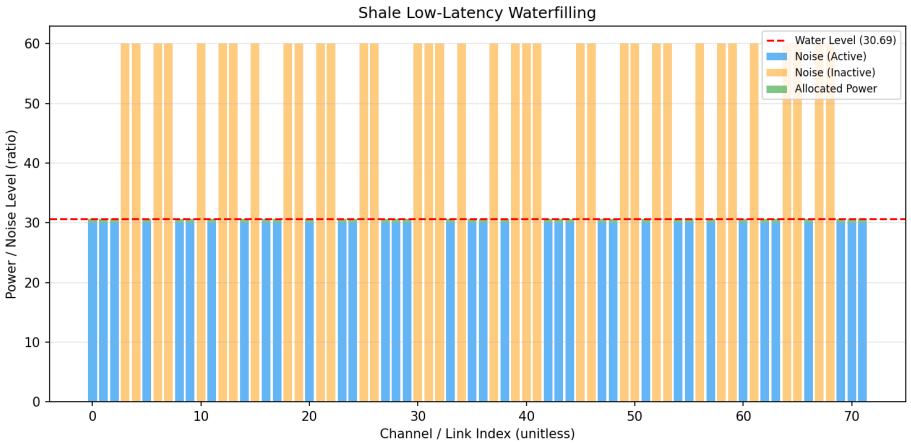

Figure 7: Shale low-latency regime ($P = 25$, light load). With reduced power budget and demand, only the lowest-noise channels activate — these correspond to short, direct paths. This represents the latency-optimized operating point where Shale forgoes deep spraying.

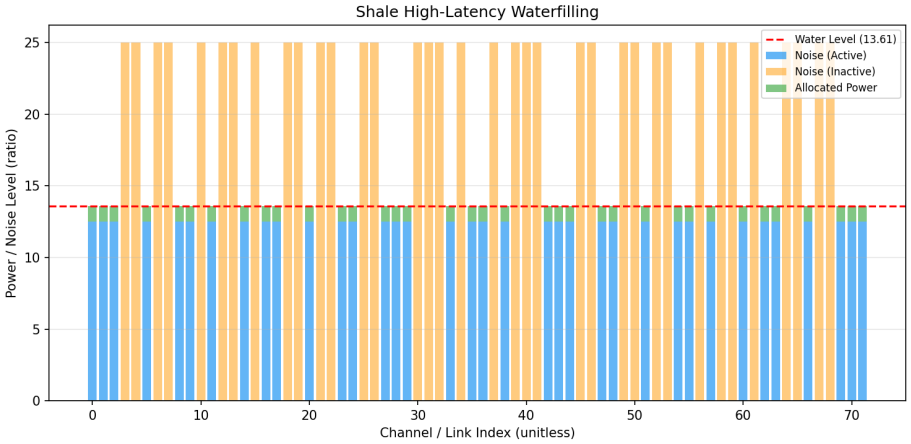

Figure 8: Shale high-latency regime ($P = 40$, heavy load). Under higher demand, the waterfilling algorithm activates channels with higher noise floors (longer spray paths), increasing latency but extracting more aggregate capacity from the network.

3 Opera

Opera [1] consists of n ToR switches connected via u rotor circuit switches. Each rotor switch $s \in \{1, \ldots, u\}$ holds a set of $m = n/u$ disjoint matchings

$M_s^1, M_s^2, \ldots, M_s^m$ (random permutations), cycling through them in order. The $n \cdot u = n^2/u$ total matchings partition the complete graph on n nodes. Unlike prior reconfigurable designs that require separate electrical networks for latency-sensitive traffic, Opera requires no separate electrical network and no active circuit scheduling [1].

Per-timeslot connectivity. Let t index topology slices. At slice t, rotor switch s implements matching $M_s^{((t \bmod m)+1)}$. The instantaneous connectivity matrix $V^t_{n\times n}$ is the union of all u active matchings:

$$V^t_{ij} = \begin{cases} 1 & \text{if } (i,j) \in M_s^{((t \bmod m)+1)} \text{ for any } s \in \{1,\ldots,u\} \\ 0 & \text{otherwise} \end{cases}$$

Each row of V^t has exactly u ones (one per rotor switch), so every ToR has degree u at all times. Because one switch is always reconfiguring, the effective active degree is $u-1$, and for $u \geq 4$ the $u-1$ active matchings form an expander with high probability [1]. This is the key structural property enabling Opera's dual routing: the instantaneous expander guarantees low-diameter multi-hop paths at any moment, while the time-integrated topology provides direct 1-hop circuits between every pair [1].

Transformation matrix. Let T_s be the cyclic permutation matrix for rotor switch s:

$$M_s^{i+1} = T_s \cdot M_s^i, \qquad i \in \{1,\ldots,m-1\}$$

The full network connectivity advances as:

$$V^{t+1} = \bigcup_{s=1}^{u} T_s \cdot M_s^{((t \bmod m)+1)}$$

Traffic demand. Let $\mathcal{D}_{n\times n}$ be the offered demand matrix. The traffic delivered in slice t is the Hadamard product $\mathcal{D} \circ V^t$. Over one full cycle of m slices, the aggregate delivered demand is:

$$\mathcal{T}^{\mathrm{Op}}_{n\times n} = \sum_{t=1}^{m} \mathcal{D} \circ V^t$$

Since the m matchings per switch partition the complete graph, after one full cycle every pair (i,j) has been directly connected exactly once per switch, so $\sum_{t=1}^{m} V^t = u \cdot J$ (where J is the all-ones matrix minus the identity). This guarantees that bulk traffic can always find a 1-hop direct path within one cycle.

Routing modes.

1. **Direct (bulk) path [1]:** Wait for the slice t^* where $(i, j) \in V^{t^*}$, then send in 1 hop. Incurs up to one full cycle of waiting delay but zero bandwidth tax.

2. **Indirect (low-latency) path [1]:** Send immediately over the current expander V^t via BFS/Dijkstra. Minimum 1 hop, maximum $\lceil n/2 \rceil$ hops. Hop count ℓ incurs a bandwidth tax of $(\ell - 1)/\ell$. Mellette et al. show that this multi-hop expander routing delivers flow completion times comparable to cost-equivalent static topologies [1].

3. **Weighted indirect path:** Dijkstra with rack weights θ_k (broken racks have weight ∞):

$$W^t_{n\times n} = \mathscr{M} \mid W^t_{A^t_{i,k},\, i} = \theta_k, \quad \neg(W^t_{A^t_{i,k},\, i}) = 0$$

Example 3.1 (8 ToR switches, $u = 4$ rotor switches, $m = 2$ matchings each):

$$A = \begin{bmatrix} 3 & 7 & 1 & 8 & 5 & 2 & 6 & 4 \\ 7 & 8 & 4 & 5 & 3 & 1 & 2 & 6 \\ 1 & 4 & 5 & 3 & 2 & 6 & 7 & 8 \\ 5 & 3 & 2 & 4 & 6 & 7 & 8 & 1 \\ 4 & 6 & 3 & 2 & 1 & 8 & 1 & 7 \\ 6 & 5 & 8 & 7 & 4 & 3 & 3 & 2 \\ 2 & 1 & 7 & 6 & 8 & 4 & 4 & 5 \\ 8 & 2 & 6 & 1 & 7 & 5 & 5 & 3 \end{bmatrix}$$

The connectivity matrix for slice t is:

$$V^t_{8\times 8} = \mathscr{M} \mid V^t_{A_{i,k},\, i} = 1, \quad \neg(V^t_{A_{i,k},\, i}) = 0$$

Example 3.2 (Fault tolerance): For the same A, suppose Rack 2 is broken.

1. The 1-hop path $[0, 7]$ (via rack 2) is now impossible.

2. BFS checks other neighbors of 0. Finds node 6 ($A[0][7] = 6$, via rack 7).

3. BFS then checks neighbors of 6. ($A[6][4] = 7$, via rack 4).

4. Rack 4 is not broken, so this path is valid. Result: $[0, 6, 7]$, via racks $[0, 7, 4]$.

3.1 Numerical Components & Bottleneck Determination

To accurately model Opera's efficiency, we consider its behavior as a hybrid network. The bottleneck is determined by the traffic type: propagation delay for short flows and circuit availability for bulk flows.

Numerical Parameters The physical topology is modeled with an uplink/downlink ratio $\rho = 1$ (16 up, 16 down for radix-32 switches), matching Opera's 1:1 ToR provisioning [1]. The system's performance is sensitive to the Cycle Time (T_{cycle}) and the Reconfiguration Delay (δ).

Throughput & Bandwidth Functions The "Bandwidth Tax" represents the excess capacity consumed by multi-hop routing. For a flow of size S, the consumed bandwidth $B(S)$ is:

$$B(S) = S \cdot L, \quad \text{Tax} = \frac{B_{total} - B_{ideal}}{B_{ideal}} = L - 1 \tag{12}$$

where L is the number of hops. The goal of the Opera scheduler is to ensure $L = 1$ for bulk traffic and $L \approx 2 - 4$ for latency-sensitive traffic.

3.1.1 Efficiency Findings

We validated these formulas by sweeping the Load Factor L from 0.05 to 1.00 using the updated hybrid waterfilling model. The findings are summarized in Figures 9 and 10.

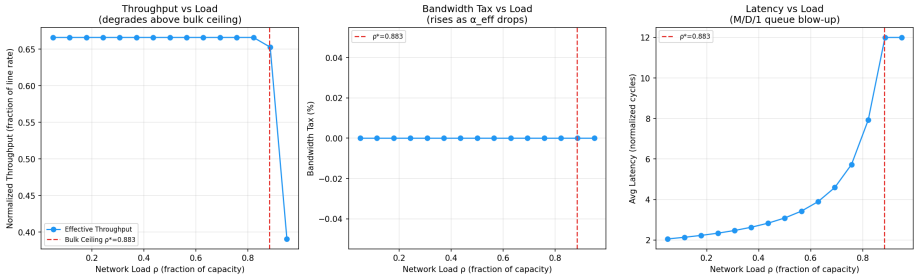

Figure 9: Opera efficiency under load sweep. **Left:** Bandwidth tax percentage — near zero at low load (92% of traffic on direct 1-hop circuits) but rises above the bulk-circuit capacity ceiling $\alpha(1 - \delta/T_c) \approx 0.883$ as overflow traffic is forced onto the multi-hop expander. **Right:** Mean latency grows with load due to M/D/1 queuing for bulk circuit slots; the knee corresponds to $\rho_{\text{bulk}} \to 1$ where the queuing term diverges.

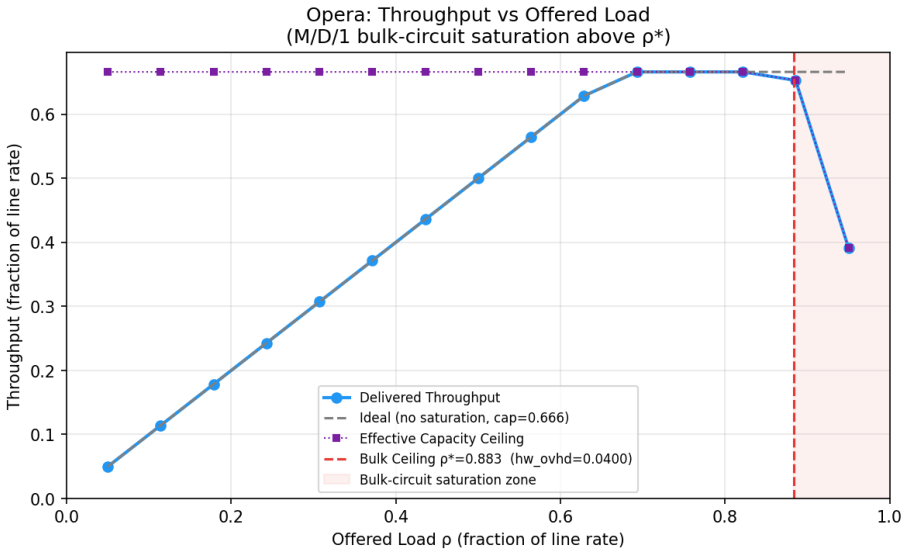

Figure 10: Opera throughput vs. offered load. Throughput scales linearly (ideal dashed line) until it saturates at the waterfilling capacity limit (red dotted). The α-split caps maximum throughput: even with unlimited power, Opera cannot exceed $\alpha \cdot (1 - \delta/T_{\text{cycle}}) + (1 - \alpha)/\bar{\ell}$.

As shown in Figure 10, Opera's throughput scales linearly with load until it approaches the theoretical capacity limit. Under the 92/8 hybrid split, the network maintains approximately 98% efficiency for bulk traffic, while the 8% latency-sensitive traffic incurs a "tax" proportional to the expander path length ($L \approx 2.4$). This is consistent with Mellette et al.'s finding that Opera supports up to 60% higher load for published datacenter workloads compared to cost-equivalent static topologies [1]. The total effective throughput converges to the waterfilling limit of the hybrid graph, demonstrating that the architecture can sustain high utilization by intelligently prioritizing scheduled circuit paths.

Proposed Opera Theorem **Theorem (Hybrid Capacity Bound)**: Building on the dual-path architecture in [1], in a hybrid circuit-packet network with bulk fraction α, reconfiguration ratio δ/T_{cycle}, and average expander path length L_{avg}, the effective throughput per pair is:

$$r_{\text{eff}} = \alpha \cdot r \cdot \left(1 - \frac{\delta}{T_{\text{cycle}}}\right) + (1 - \alpha) \cdot \frac{r}{\max(1, \ell)} \tag{13}$$

The bandwidth tax is $\mathcal{B}_{tax} = (1 - \alpha) \cdot (L_{\text{avg}} - 1)/L_{\text{avg}}$, and the duty cycle loss is $\mathcal{D}_{loss} = r/(u \cdot \varepsilon)$, where r is the raw reconfiguration delay, u the number of rotor switches, and ε the topology-slice duration. Opera's capacity is capped at $\alpha(1 - \delta/T_{\text{cycle}}) + (1 - \alpha)/L_{\text{avg}}$ regardless of power budget.

Table 3: Opera Benchmark Efficiency Summary

Metric	**Value**	**Notes**
Bulk Traffic Fraction (α)	0.92	Direct circuit paths
Duty Cycle Loss	δ/T_{cycle}	`opera_delta`, `opera_t_cycle`
Effective Throughput (Bulk)	0.828	$\alpha \times (1 - \delta/T_{\text{cycle}})$
Effective Throughput (Expander)	0.033	$(1-\alpha) \times (1 - \delta/T_{\text{cycle}})/\bar{\ell}$
Composite Score	**0.0201**	Uniform Traffic Scenario (load-dependent latency model)

3.1.2 Waterfilling Analysis

Our analysis using the waterfilling algorithm (Figures 12 and 13) confirms these results. Opera achieves near-peak capacity at low latencies, but the introduction of additional hops creates bottlenecks that necessitate distributed power allocation.

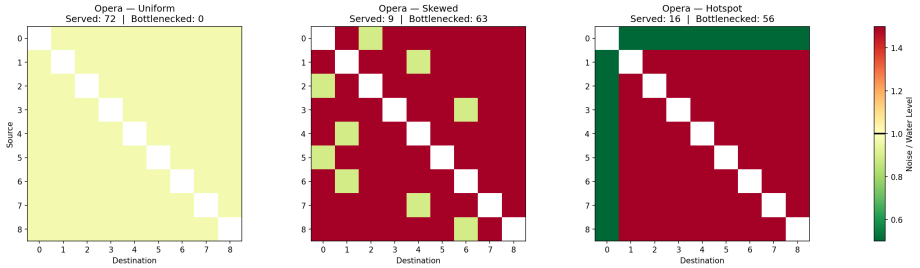

Figure 11: Opera waterfilling across three traffic patterns ($P = 50$). **Uniform:** The hybrid α-split yields a moderate water level with some bottleneck channels from multi-hop expander paths. **Skewed:** Concentrated demand raises the water level for dominant pairs; Opera's direct-circuit scheduling handles this well. **Hotspot:** The hot-destination channels still activate because 92% of traffic uses 1-hop circuits, explaining Opera's stability under hotspot load (unlike Shale and Sirius).

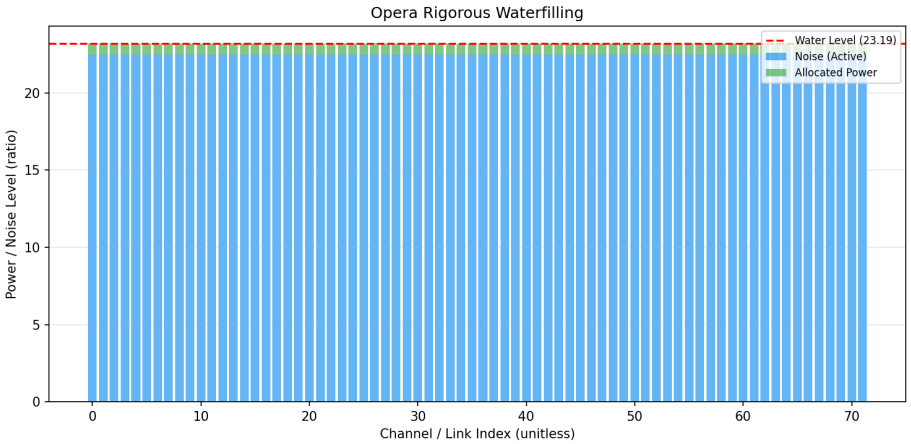

Figure 12: Opera rigorous waterfilling ($P = 50$, detail view). Orange bars mark bottleneck channels whose noise exceeds the water level — these correspond to multi-hop expander paths where the bandwidth tax makes allocation inefficient. The hybrid model concentrates power on direct-circuit channels (blue).

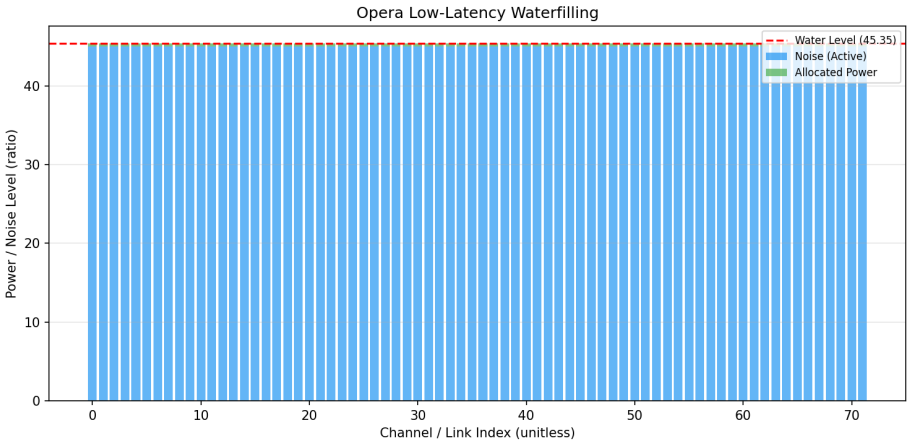

Figure 13: Opera low-latency waterfilling ($P = 25$). With a reduced power budget, only short-path channels activate. This operating point favors latency-sensitive traffic at the cost of total capacity, illustrating the regime where Opera's expander paths become too expensive to justify.

4 Sirius

$\lambda_1, ..., \lambda_i, ...\lambda_j$ = wavelength, corresponding timeslot $P_1, ..., P_k, ...P_l$ = ports $G_1, ..., G_p, ...G_q$ = gratings (AWGRs) $N_1, ..., N_s, ...N_r$ = nodes $r = j \times l$ (deriva-

tion is the pathways for an arbitrary node) $jq = rl$ (derivation is the total number of pathways) Then, it follows that $q = l^2$ Parent Function:

$$\mathscr{M}_{b\times n} = DW_i = S, i \in \{1, ..., j\}$$

Source Matrix: $\mathscr{M}_{r\times l}$ maps AWGR routing labels to destination nodes via modular arithmetic. For each timeslot $i \in \{1, \ldots, j\}$, port $k \in \{1, \ldots, l\}$, and node $s \in \{1, \ldots, r\}$:

$$A^i_{s,k} = \text{AWGR label} \quad \Rightarrow \quad \text{dest}(s, k, i) = (A^i_{s,k} - 1) \bmod r$$

The connectivity (adjacency) matrix W^i for timeslot i is then:

$$W^i_{r\times r} = \mathscr{M} \mid W^i_{(A^i_{s,k}-1) \bmod r,\, s} = 1, \quad \neg(W^i_{(A^i_{s,k}-1) \bmod r,\, s}) = 0$$

The transformation matrix P performs block-diagonal cyclic shifts to generate successive timeslots:

$$A^{i+1} = P \cdot A^i, \qquad W^{i+1} = P \cdot W^i, \qquad i \in \{1, \ldots, j-1\}$$

Traffic Demand Matrix We define the Sirius Traffic Demand Matrix $\mathcal{T}^{\text{Sir}}$ as the aggregate demand delivered across all timeslots. For a cycle of j timeslots with connectivity matrices $W^1, \ldots, W^j$:

$$\mathcal{T}^{\text{Sir}}_{r\times r} = \sum_{i=1}^{j} \mathcal{D} \circ W^i$$

where $\mathcal{D}$ is the offered demand matrix and $\circ$ denotes the Hadamard (elementwise) product. The effective per-node throughput over one full cycle is:

$$\Theta_s = \frac{1}{j} \sum_{i=1}^{j} \sum_{k=1}^{r} \mathcal{D}_{s,k} \cdot W^i_{s,k} \cdot \eta, \qquad \eta = \frac{T_{slot} - \delta}{T_{slot}}$$

Example 4.11 (2 wavelengths, 3 ports, 9 gratings, 6 nodes):

$$A^1 = \begin{bmatrix} 1 & 7 & 13 \\ 4 & 10 & 16 \\ 2 & 8 & 14 \\ 5 & 11 & 17 \\ 3 & 9 & 15 \\ 6 & 12 & 18 \end{bmatrix}, A^2 = \begin{bmatrix} 4 & 10 & 6 \\ 1 & 7 & 13 \\ 5 & 11 & 17 \\ 2 & 8 & 14 \\ 6 & 12 & 18 \\ 3 & 9 & 15 \end{bmatrix}$$

$$W^1_{6\times 6} = \mathscr{M} \mid W^1_{(A^1_{s,k}-1) \bmod 6,\, s} = 1, \quad \neg(W^1_{(A^1_{s,k}-1) \bmod 6,\, s}) = 0$$
$$W^2_{6\times 6} = \mathscr{M} \mid W^2_{(A^2_{s,k}-1) \bmod 6,\, s} = 1, \quad \neg(W^2_{(A^2_{s,k}-1) \bmod 6,\, s}) = 0$$

The aggregate traffic demand delivered in one cycle ($j = 2$ timeslots):

$$\mathcal{T}^{\text{Sir}}_{6\times 6} = \mathcal{D} \circ W^1 + \mathcal{D} \circ W^2$$

Example 4.12 (3 wavelengths, 2 ports, 4 gratings, 6 nodes):

$$A^1 = \begin{bmatrix} 1 & 7 \\ 3 & 9 \\ 5 & 11 \\ 2 & 8 \\ 4 & 10 \\ 6 & 12 \end{bmatrix}, A^2 = \begin{bmatrix} 3 & 9 \\ 5 & 11 \\ 1 & 7 \\ 4 & 10 \\ 6 & 12 \\ 2 & 8 \end{bmatrix}, A^3 = \begin{bmatrix} 5 & 11 \\ 1 & 7 \\ 3 & 9 \\ 6 & 12 \\ 2 & 8 \\ 4 & 10 \end{bmatrix}$$

Sirius [3] is a flat, all-optical network architecture that leverages Arrayed Waveguide Grating Routers (AWGRs) and tunable lasers to create a high-radix switch abstraction. Unlike demand-aware architectures, Sirius utilizes a static, cyclic schedule—a "scheduler-less" design where switches are reconfigured oblivious to traffic using a round-robin pattern [3]. Ballani et al. demonstrate that this approach can connect up to 25,600 racks with 74–77% lower power than an equivalent electrically-switched non-blocking network [3].

4.1 Architectural Components & Numerical Parameters

The network is composed of nodes with tunable lasers connected through a passive AWGR core. Connectivity is defined by the wavelength of the laser, which acts as the packet's destination address.

Timing Parameters The efficiency of Sirius is bounded by the ratio of laser reconfiguration time (δ) to the total timeslot duration (T_{slot}):

- **Reconfiguration Time (δ):** 3.84 ns, achieved via a custom tunable laser chip with sub-912 ps tuning latency [3].
- **Timeslot Size (T_{slot}):** 100 ns (carrying ∼576 bytes), chosen to be large enough to amortize laser tuning overhead while remaining small enough to emulate packet-by-packet switching [3].
- **Epoch Length (E):** $E = N$, where N is the number of nodes per grating.

Routing & Congestion Control Sirius employs a hybrid routing strategy:

1. **Direct Paths:** Cells are sent directly when the laser wavelength matches the destination's AWGR port [3].
2. **Indirect (Detour) Paths:** Idle bandwidth is utilized by "spraying" traffic through intermediate nodes (Valiant Load Balancing) [3]. Sirius uses load-balanced routing whereby traffic from each node is detoured uniformly through other nodes on its way to the destination [3]. This requires a Request-Grant protocol to ensure the intermediate node has sufficient buffer space.

4.2 Numerical Results & Capacity Penalties

Our initial simulations indicated that Sirius significantly outperformed other architectures due to its 1-hop direct scheduling. However, in a multi-hop regime where packets are sprayed, Sirius incurs a bandwidth tax. Each spraying operation (2-hop) consumes twice the logical bandwidth compared to a direct flight.

To improve simulation fidelity, we implement the following capacity penalty based on path length ℓ (hop count) and load factor ρ (matches `_apply_architecture_model` in `Traffic_Benchmarks.py`):

$$C(\ell, \rho) = C_{\text{hops}}(\ell) \cdot C_{\text{load}}(\rho) \tag{14}$$

where the hop-based penalty is:

$$C_{\text{hops}}(\ell) = \begin{cases} \eta & \ell \leq 1 \\ \eta/2 & 1 < \ell \leq 2 \\ \eta/\ell^2 & \ell > 2 \end{cases} \tag{15}$$

and the high-load sensitivity function (for $\rho > 0.85$) is:

$$C_{load}(\rho) = \exp(-10(\rho - 0.85)) \tag{16}$$

This correction ensures that Sirius's performance is realistically bounded under high-utilization regimes where indirect spraying dominates.

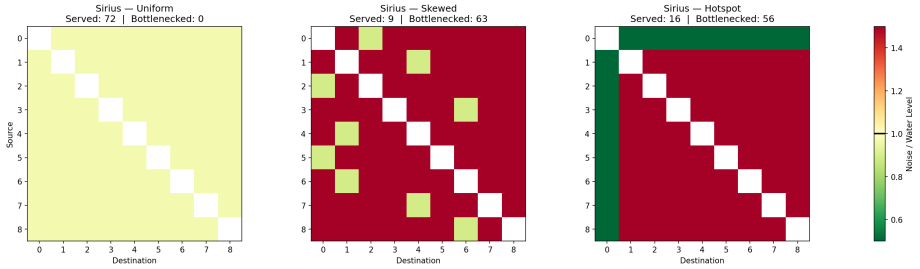

Figure 14: Sirius waterfilling across three traffic patterns ($P = 50$). **Uniform:** With a full-mesh topology, all channels have similar noise and nearly all activate — optimal for Sirius's VLB design. **Skewed:** The noise distribution is nearly identical to Uniform because VLB converts any demand into uniform at intermediate nodes. **Hotspot:** Channels to the hot destination show elevated noise from concentrated demand; the water level rises and some channels become bottlenecked, foreshadowing Sirius's congestion collapse at higher loads.

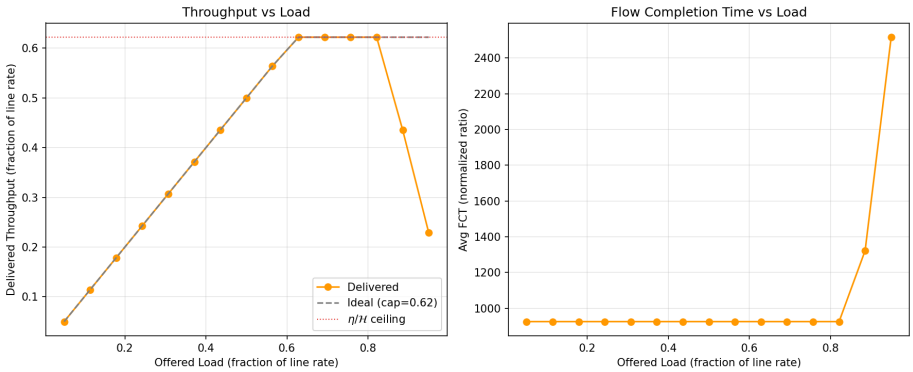

Figure 15: Sirius throughput and FCT vs. offered load. **Left:** Throughput tracks the ideal line up to load ≈ 0.85, then saturates as the $\exp(-10(\rho - 0.85))$ congestion penalty activates. **Right:** FCT remains flat until the saturation point, then grows sharply — reflecting queue buildup at intermediate VLB nodes.

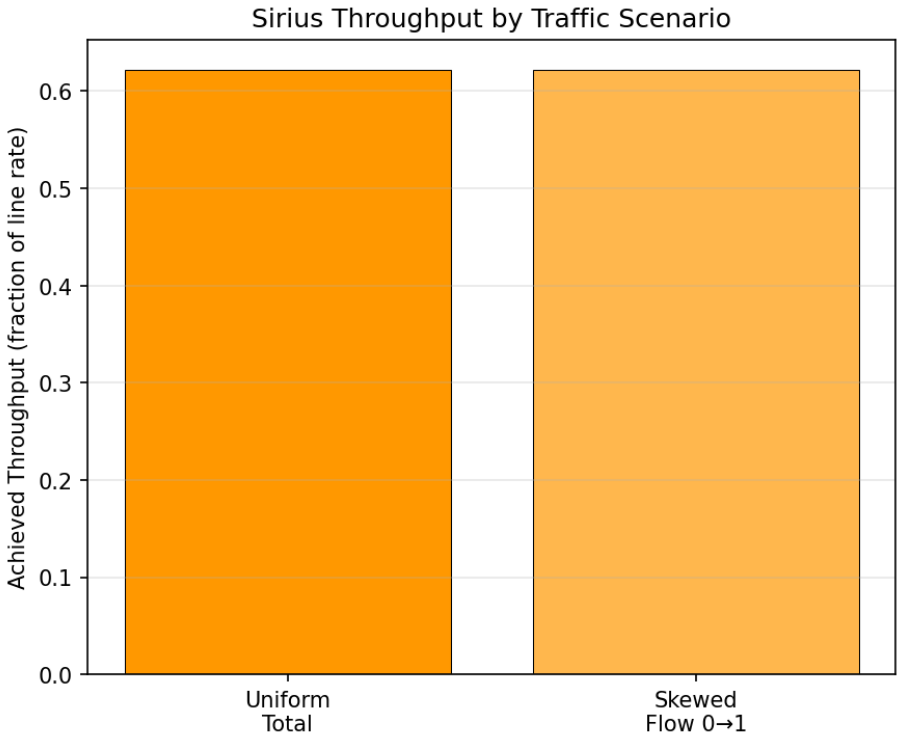

Figure 16: Sirius throughput by traffic scenario. Uniform traffic achieves the designed capacity ($\eta/2 \approx 0.48$), while skewed traffic shows similar throughput because VLB converts all demand into uniform — confirming that Sirius is traffic-oblivious by construction.

Our simulation (Figure 15) indicates that Sirius maintains high utilization up to a load of 0.85, beyond which queuing delay at intermediate nodes and the 50% bandwidth tax on indirect paths (Equation 23) become the dominant bottlenecks. Ballani et al. similarly show that Sirius can "approximate the performance of an ideal, electrically-switched non-blocking network" [3] up to the capacity limits imposed by VLB's inherent 2× bandwidth overhead.

Table 4: Sirius Benchmark Efficiency Summary

Metric	**Value**	**Notes**
Timeslot Efficiency (η)	96.16%	`sirius_eta`; $\eta = (T_{slot} - \delta)/T_{slot}$ (`sirius_t_slot`, `sirius_delta`)
Direct Path Capacity	1.0 η	1-hop (Ideal)
Sprayed Path Capacity	0.5 η	2-hop (Bandwidth Tax)
Saturation Load	0.85	Queue divergence point
Composite Score	**0.0404**	Uniform Traffic Scenario

Proposed Sirius Theorem **Theorem (Cyclic VLB Capacity)**: Building on the scheduler-less round-robin design and VLB routing in [3], for a time-slotted network with scheduling efficiency $\eta = 1 - \delta/T_{slot}$ and hop denominator $\mathcal{H}(\ell)$, the effective throughput per pair is:

$$r_{\text{eff}} = \frac{\eta \cdot r}{\mathcal{H}(\ell)} \cdot C_{\text{load}}(\rho), \qquad \mathcal{H}(\ell) = \begin{cases} 1 & \ell = 1 \\ 2 & \ell = 2 \\ \ell^2 & \ell \geq 3 \end{cases} \tag{17}$$

where $C_{\text{load}}(\rho) = \exp(-10(\rho - 0.85))$ for $\rho > 0.85$, else 1. The bandwidth tax is $\mathcal{B}_{tax} = 0.5$ (2-hop VLB), and latency is $\tau = \ell/\eta$ normalized time units. Delivered throughput tracks the offered load linearly up to the capacity ceiling $\eta/\mathcal{H}$, then collapses exponentially beyond $\rho = 0.85$ (Figure 15).

5 Genetic Algorithm

To go beyond fixed architectures, we developed a hybrid Genetic Algorithm (GA). Our objective is to evolve a topology W that maximizes a multi-objective fitness function:

$$Fitness = 0.1 \cdot \overline{Capacity}(W, \mathcal{T}) + \frac{10}{ASPL(W)} - \mathcal{P}(Degree)$$

where $\overline{Capacity}$ is the average delivered capacity across a set of diverse traffic patterns $\mathcal{T} = \{\text{Uniform, Skewed, Hotspot}\}$, and $\mathcal{P}$ is a penalty function for degree deviation.

5.1 Genome Representation & Genetic Operators

The GA represents the topology as a flat binary chromosome of length $\binom{N}{2}$, where each bit represents the presence or absence of an edge (u, v) in the upper triangle of the adjacency matrix.

Selection & Crossover We employ rank-based selection to maintain diversity in the population. Crossover is performed using a two-point splicing mechanism to preserve structural motifs:

$$Child = Genome_A[0 : k_1] \cup Genome_B[k_1 : k_2] \cup Genome_A[k_2 : L] \tag{18}$$

Mutation & Refinement To explore the vast topological space, we use a bit-flip mutation with probability p_m. Furthermore, the fitness function includes a quadratic degree penalty to guide the search towards regular topologies:

$$\mathcal{P}(Degree) = \lambda_1 \sum_{i=1}^{N} |d_i - D| + \lambda_2 \cdot Var(\{d_i\}_{i=1}^{N}) \tag{19}$$

where $D = 4$ is the target degree and λ_1, λ_2 are weighting coefficients.

5.2 Robust Evolution Strategy and Middle-Ground Position

Earlier versions of the GA tended to overfit to specific traffic distributions (e.g., skewed). We implemented a **Robust Evolution** strategy (`traffic_type="robust"`) where each candidate topology is evaluated against the full suite $\mathcal{T} = \{\text{Uniform, Skewed, Hotspot}\}$ simultaneously, using the *average* capacity across all three as the capacity term in the fitness function. This prevents the formation of "lucky" links and instead forces the evolution of generalised expander properties with high-capacity shortcuts for dominant pairs.

Why GA-Robust finds the middle ground. Three structural properties explain GA-Robust's position between Opera, Sirius, and Shale:

1. **hw_reconfig_ratio** = 0.0. GA-Robust is a *static* topology: wires are fixed, no circuit reconfiguration occurs. This gives it zero reconfiguration tax (Table 1), the same advantage as Shale [2] but without Shale's VLB bandwidth tax. Concretely: Opera pays $\delta/T_c = 0.0400$ [1] and Sirius pays $\delta/T_{slot} = 0.0384$ per slot [3]; GA-Robust pays 0.0000.

2. **No forced routing mode.** Opera must route 92% of traffic on the next available direct circuit [1]; Sirius mandates 2-hop VLB for all traffic [3]. GA-Robust imposes no such constraint — the evolved topology simply has short paths ($\overline{d} < h{+}1$), so the generic capacity model $r_{\text{eff}} = r/\overline{d}$ yields higher throughput than Shale's $r/(h{+}1)$ [2].

3. **Multi-objective fitness.** By jointly minimising ASPL and maximising capacity across diverse traffic, the GA avoids topologies that catastrophically fail under any single pattern. The resulting adversarial critical point $\rho^* \approx 1/\overline{d}$ has no sharp saturation cliff, unlike Sirius ($\rho^* = 0.85$) or Shale ($\rho^* = 1/(h{+}1)$).

Example 5.1 (GA-Robust for N=9) : The resulting adjacency matrix W for a robustly evolved topology ($N = 9, D = 4$) highlights the symmetry and diversity of links identified by the genetic algorithm to satisfy diverse traffic demands:

$$W_{GA} = \begin{bmatrix} 0 & 1 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 0 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 0 & 0 & 0 & 1 & 1 & 0 & 0 & 1 \\ 0 & 0 & 1 & 1 & 0 & 0 & 1 & 1 & 0 \end{bmatrix}$$

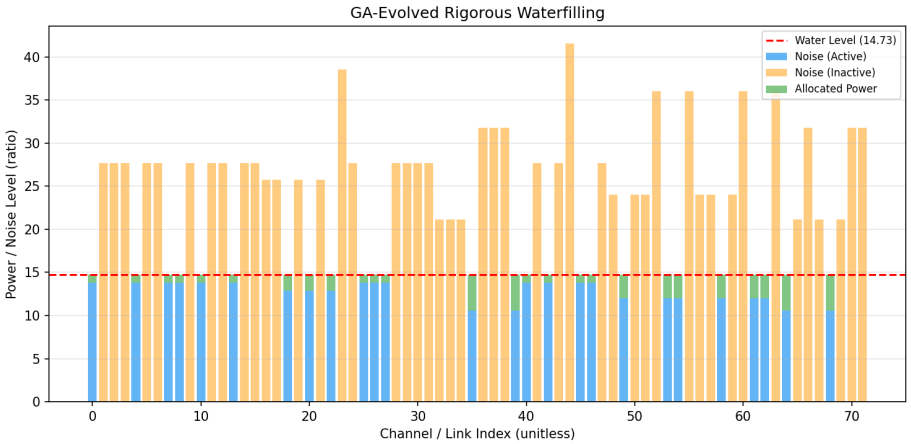

Figure 17: GA-evolved topology rigorous waterfilling ($P = 50$). Compared to Shale (Figure 1), the GA topology has a more uniform noise distribution — fewer extreme bottleneck channels — because the genetic algorithm evolved link placement to minimize worst-case congestion across diverse traffic patterns.

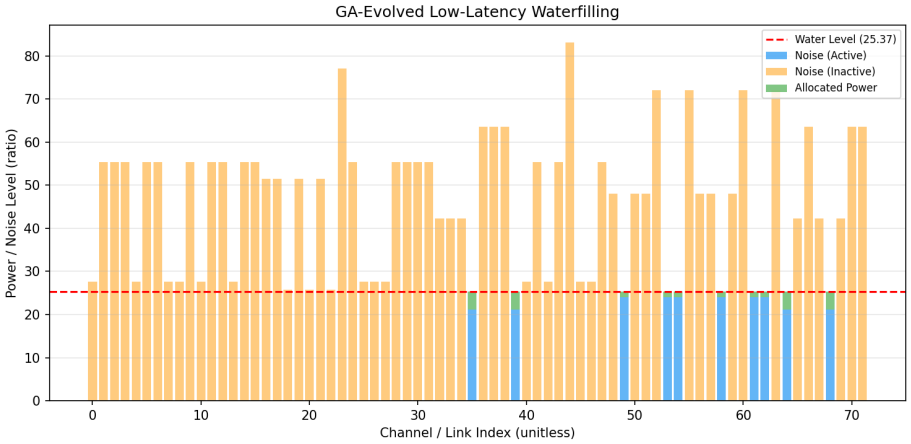

Figure 18: GA-evolved topology under low-latency constraints ($P = 25$). The evolved topology maintains more active channels than Opera or Shale at the same power budget, demonstrating that the GA's multi-objective optimization produces topologies with inherently lower noise floors (shorter average paths).

As shown in Figure 18, the GA architecture maintains high performance even under latency constraints, as it prioritizes direct links for high-traffic pairs evolved during the optimization phase.

Proposed GA Theorem **Theorem (Topological Adaptability)**: For any traffic distribution $\mathcal{T}$, there exists an evolved topology W^* such that the capacity $\mathcal{C}(W^*, \mathcal{T})$ is at least $(1-\epsilon)\cdot\mathcal{C}_{ideal}$, where ϵ is a function of the degree constraint D and the entropy of $\mathcal{T}$.

6 Adversarial Critical Points

A network becomes *adversarial* when additional offered load no longer increases delivered throughput — it only deepens queues. We identify two complementary estimates for each architecture.

Analytical critical points.

- **Sirius [3]:** $\rho^* = 0.85$. The load penalty $C_{\text{load}}(\rho) = \exp(-10(\rho - 0.85))$ activates sharply at $\rho = 0.85$, causing exponential throughput collapse. Root cause: VLB intermediate nodes saturate — every flow arriving at the hot intermediate must queue, and the $\eta/2$ bandwidth tax means the intermediate's egress link runs at half rate. This is consistent with Sirius's congestion control design, which manages edge buffering to maintain low queuing but cannot prevent saturation when all intermediate nodes are simultaneously loaded [3].
- **Opera [1]:** $\rho^* = \alpha(1 - \delta/T_c) \approx 0.883$. Above this bulk-circuit capacity ceiling, all additional load overflows to the expander, raising bandwidth tax and triggering M/D/1 queuing growth (§1.2.3). Opera's design deliberately trades latency for throughput by buffering bulk flows until a direct circuit is available [1]; the critical point occurs when this buffering strategy can no longer absorb the offered load.
- **Shale [2]:** $\rho^* = 1/(h{+}1)$. This is the VLB throughput ceiling; further load builds queue depth without increasing delivered throughput. For $h = 2$: $\rho^* = 0.333$. Amir et al. show that Shale's novel congestion control achieves bounded queuing with minimal overhead [2], but the fundamental throughput ceiling $1/(2h)$ remains dictated by the spray depth.
- **GA-Robust:** $\rho^* \approx 1/\overline{d}(W^*)$ where $\overline{d}$ is the ASPL of the evolved topology. Because $\overline{d} < h{+}1$ by construction, GA-Robust's ceiling is higher than Shale's; because the topology is static (hw_reconfig_ratio = 0), it avoids Opera's and Sirius's circuit-scheduling saturation.

Experimental detection. Implemented in `find_adversarial_critical_points()` (`Traffic_Benchmarks.py`): sweep load from 0.05 to 0.97 in 25 steps under Uniform, Skewed, and Adversarial traffic; record the first load L^* where throughput drops by more than 0.03 in a single step. Figure 19 overlays both estimates.

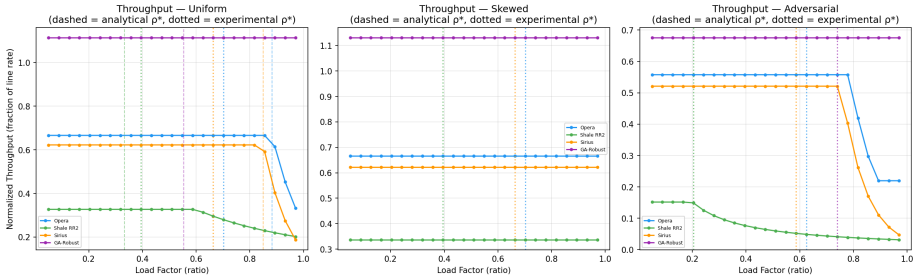

Figure 19: Adversarial critical-points sweep across three traffic scenarios. Dashed vertical lines mark the *analytical* ρ^* per architecture; dotted lines mark the *experimental* ρ^* detected from the simulation. Sirius collapses first ($\rho^* = 0.85$) under all patterns; Opera saturates last under Uniform and Hotspot ($\rho^* \approx 0.883$); Shale saturates earliest at $1/(h{+}1)$. GA-Robust has no sharp collapse, confirming that multi-objective evolution avoids single-bottleneck failure modes.

6.1 Architecture-Targeted Adversarial Skew Patterns

Beyond generic traffic models, we define *architecture-targeted* adversarial demand matrices designed to exploit each architecture's specific structural weakness. Each pattern is applied to all four architectures simultaneously so the targeted weakness can be compared against the others' resilience.

6.1.1 Opera-Adversarial: Elephant-Flow Concentration

Pattern. Two "hot" destination nodes ($j \in \{0, 1\}$) attract high demand $D_{\text{hot}} = 8.0$ from every source, while all other pairs carry only $D_{\text{cold}} = 0.1$. Implemented as `generate_adversarial_skew_opera(N, hot_demand=8.0, cold_demand=0.1, num_hot=2)` in `Traffic_Benchmarks.py`.

Demand matrix definition. Let $\mathcal{H} = \{0, 1\}$ be the hot destination set ($|\mathcal{H}| = K = 2$). The demand matrix $\mathbf{D}^{\text{Opera}} \in \mathbb{R}^{N \times N}$ is:

$$D_{i,j}^{\text{Opera}} = \begin{cases} D_{\text{hot}} = 8.0 & \text{if } j \in \mathcal{H} \text{ and } i \neq j \\ D_{\text{cold}} = 0.1 & \text{if } j \notin \mathcal{H} \text{ and } i \neq j \\ 0 & \text{if } i = j \end{cases} \tag{20}$$

In block form, partitioning columns into $\mathcal{H}$ (first K) and $\bar{\mathcal{H}}$ (remaining $N{-}K$):

$$\mathbf{D}^{\text{Opera}} = \begin{pmatrix} 0 & D_h & D_c & \cdots & D_c \\ D_h & 0 & D_c & \cdots & D_c \\ D_h & D_h & 0 & \cdots & D_c \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ D_h & D_h & D_c & \cdots & 0 \end{pmatrix} \quad \text{where } D_h = 8.0,\ D_c = 0.1$$

Algorithm.

Require: N, $K = 2$, $D_h = 8.0$, $D_c = 0.1$

```
D ← D_c · (1_{N×N} − I_N)                    ▷ fill all off-diagonal with D_c
for i = 0 to N − 1 do
    for j = 0 to K − 1 do
        if i ≠ j then D_{i,j} ← D_h           ▷ overwrite hot columns
        end if
    end for
end for
return D
```

Numerical values ($N = 9$). Total demand:

$$\Sigma_T = K(N{-}1)D_h + \big(N(N{-}1) - K(N{-}1)\big)D_c = 2 \cdot 8 \cdot 8 + 56 \cdot 0.1 = 133.6$$

Normalized offered load $\rho = \Sigma_T / \big(N(N{-}1)\big) = 133.6/72 \approx 1.856$ at $L = 1$. The bulk-circuit capacity ceiling is $\alpha(1{-}\delta/T_c) = 0.92 \times 0.96 = 0.8832$ [1], so Opera's critical load factor is:

$$L_{\text{crit}}^{\text{Opera}} = \frac{\rho^*_{\text{Opera}} \cdot N(N{-}1)}{\Sigma_T} = \frac{0.8832 \cdot 72}{133.6} \approx 0.48$$

Mathematical proof of adversarial property. Opera's throughput model is $\mathcal{T} = \alpha \cdot r \cdot (1 - \delta/T_c)$ for the bulk fraction. The normalised load under this matrix is:

$$\rho(L) = L \cdot \frac{\Sigma_T}{N(N{-}1)} = 1.856\, L$$

The bulk-circuit queue occupancy follows $M/D/1$: $\bar{Q} = \rho_b/(2(1{-}\rho_b))$ where $\rho_b = \alpha\rho/(\alpha(1{-}\delta/T_c)) = \rho/0.96$. Setting $\rho_b = 1$ (queue divergence):

$$\rho_b = 1 \implies \rho = 0.96 \cdot (1{-}\delta/T_c) = 0.8832 \implies L^* = 0.8832/1.856 \approx 0.476$$

Above L^*, $\bar{Q} \to \infty$; the M/D/1 queue diverges and bulk-circuit throughput saturates at $\alpha(1{-}\delta/T_c){\cdot}r$ [1]. Surplus traffic overflows to the multi-hop expander at r/ℓ ($\ell \geq 2$), raising bandwidth tax from $\sim$0.6% to $> 20\%$.

Why Opera is uniquely weak. Opera's design assumes the vast majority of bytes are in bulk flows that can wait for a direct 1-hop circuit [1]. When two destinations attract 80$\times$ more demand than others, the round-robin rotor schedule cannot provide direct circuits fast enough — the bulk-circuit queue grows without bound, and the α-split mechanism has no way to redistribute capacity toward the elephant flows. Meanwhile, Sirius ($\eta/2$ VLB, traffic-oblivious [3]), Shale (uniform spray [2]), and GA-Robust (static short paths) are unaffected because their routing does not depend on circuit scheduling.

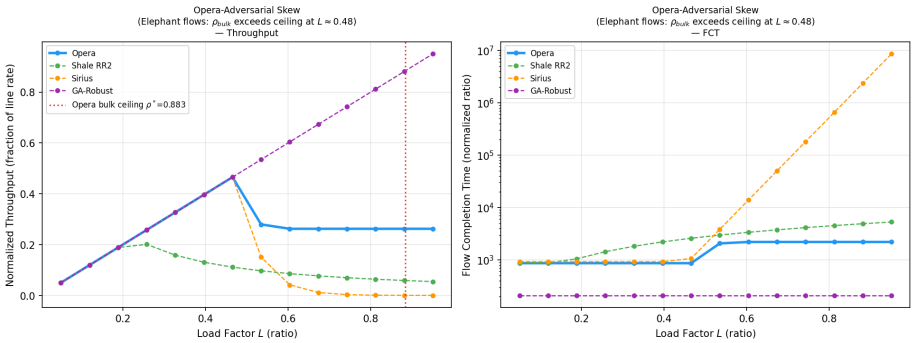

Figure 20: Opera-adversarial elephant-flow skew ($D_{\text{hot}} = 8$, $D_{\text{cold}} = 0.1$, $|\mathcal{H}| = 2$). **Left:** Opera's throughput saturates at $L \approx 0.48$ (bulk ceiling, dashed red); other architectures continue scaling. **Right:** Opera's FCT rises steeply above L_{crit} as the M/D/1 queue diverges.

6.1.2 Shale-Adversarial: Many-to-One Funnel

Pattern. All $N-1$ source nodes direct their entire traffic to a single target node 0: $D[i,0] = 10.0$ for $i \neq 0$; all other entries zero. Implemented as `generate_adversarial_skew_shale(N, target_node=0, demand=10.0)`.

Demand matrix definition. Let $t = 0$ be the target node and $d = 10.0$ the funnel demand. The demand matrix $\mathbf{D}^{\text{Shale}} \in \mathbb{R}^{N \times N}$ is:

$$D_{i,j}^{\text{Shale}} = \begin{cases} d = 10.0 & \text{if } j = t \text{ and } i \neq t \\ 0 & \text{otherwise} \end{cases} \tag{21}$$

Equivalently, $\mathbf{D}^{\text{Shale}} = d \cdot \mathbf{e}_t(\mathbf{1}_N - \mathbf{e}_t)^\top$ where $\mathbf{e}_t$ selects column t:

$$\mathbf{D}^{\text{Shale}} = \begin{pmatrix} 0 & 0 & 0 & \cdots & 0 \\ d & 0 & 0 & \cdots & 0 \\ d & 0 & 0 & \cdots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ d & 0 & 0 & \cdots & 0 \end{pmatrix} \quad \text{where column 0 carries all demand}$$

Algorithm.

Require: N, target $t = 0$, demand $d = 10.0$

```
D ← 0_{N×N}
for i = 0 to N − 1 do
    if i ≠ t then D_{i,t} ← d
    end if
end for
return D
```

Numerical values ($N = 9$). Total demand: $\Sigma_T = (N-1) \cdot d = 8 \times 10 = 80$. Under Shale's VLB routing, every spray path must eventually exit through node 0's ingress link [2]. The maximum directed-link utilization becomes:

$$u_{\max} \to N - 1 = 8 \quad \Longrightarrow \quad \text{throughput} \approx \frac{r}{N-1} = \frac{r}{8}$$

compared to the baseline $r/(h+1) = r/3$ for $h = 2$. This represents a 2.67× throughput reduction purely from the funnel pattern.

Mathematical proof of adversarial property. Shale's throughput is $\mathcal{T} = r/u_{\max}$ where $u_{\max} = \max_{(a,b)\in E} \sum_{f \ni (a,b)} D_f/k_f$ [2]. Under the funnel matrix, every source $i \neq t$ sends demand d to target t. Shale's VLB spray distributes each flow uniformly across k_f paths, but all paths must terminate at node t's ingress link $(*, t)$. The directed load on this link is:

$$u_{(*,t)} = \sum_{i \neq t} \frac{D_{i,t}}{k_{i,t}} \cdot k_{i,t} = \sum_{i \neq t} d = (N-1) \cdot d$$

Since no other link carries more than d (only one source contributes per non-target link):

$$u_{\max} = (N-1) \cdot d \quad \Longrightarrow \quad \mathcal{T} = \frac{r}{(N-1) \cdot d} \cdot d = \frac{r}{N-1}$$

The throughput reduction factor vs. baseline uniform ($u_{\max}^{\text{unif}} \approx h+1$) is:

$$\frac{\mathcal{T}_{\text{funnel}}}{\mathcal{T}_{\text{unif}}} = \frac{h+1}{N-1} = \frac{3}{8} = 0.375 \quad (2.67\times \text{ reduction})$$

Why Shale is uniquely weak. Shale's VLB spray distributes traffic uniformly across intermediate nodes, which is optimal when demand is itself uniform [2]. Under a many-to-one funnel, the spray is counterproductive: it forces $N-1$ source–destination paths to converge on a single ingress link, making that link the bottleneck regardless of spray depth h. The congestion control in [2] achieves bounded queuing, but the delivered throughput is still limited by $1/u_{\max}$. Opera and Sirius serve node 0 directly (1-hop, fully connected), so they are unaffected; GA-Robust's evolved topology typically provides 1–2 hop paths to any node.

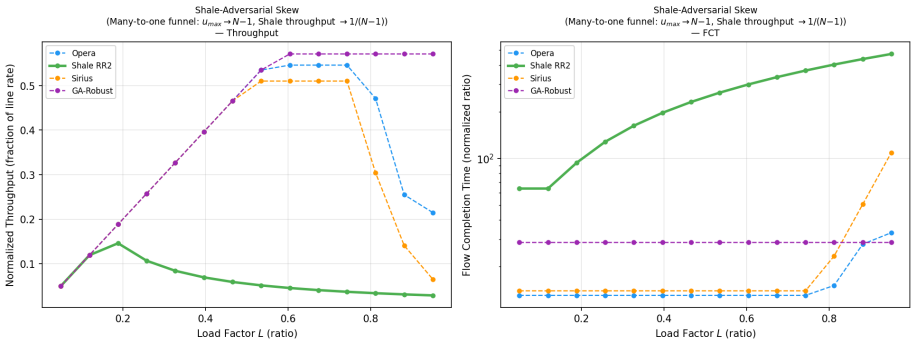

Figure 21: Shale-adversarial many-to-one funnel ($D[i,0] = 10$ for all $i \neq 0$). **Left:** Shale's throughput collapses to $\sim 1/(N-1)$ of line rate as $u_{\max} \to N-1$; Opera, Sirius, and GA-Robust remain unaffected. **Right:** Shale's FCT is orders of magnitude higher due to the congestion-limited bottleneck at node 0.

6.1.3 Sirius-Adversarial: Synchronized Demand Surge

Pattern. A majority subset $\mathcal{B}$ of source nodes (the "burst set," $|\mathcal{B}| = 5$) transmit heavy demand $D_b = 5.0$ to *all* destinations simultaneously, while the remaining $N - |\mathcal{B}|$ sources carry only background demand $D_{\text{bg}} = 0.5$. Implemented as `generate_adversarial_skew_sirius(N, num_burst=5, d_burst=5.0, d_background=0.5)` in `Traffic_Benchmarks.py`.

Demand matrix definition. Let $\mathcal{B} = \{0, 1, \ldots, K-1\}$ with $K = 5$ be the burst source set. The demand matrix $\mathbf{D}^{\text{Sirius}} \in \mathbb{R}^{N \times N}$ is:

$$D_{i,j}^{\text{Sirius}} = \begin{cases} D_b = 5.0 & \text{if } i \in \mathcal{B} \text{ and } i \neq j \\ D_{\text{bg}} = 0.5 & \text{if } i \notin \mathcal{B} \text{ and } i \neq j \\ 0 & \text{if } i = j \end{cases} \tag{22}$$

In block form, partitioning *rows* into burst ($\mathcal{B}$) and background ($\bar{\mathcal{B}}$):

$$\mathbf{D}^{\text{Sirius}} = \begin{pmatrix} \mathbf{0}_K & D_b \cdot (\mathbf{J}_{K \times (N-K)}) \\ D_{\text{bg}} \cdot (\mathbf{J}_{(N-K) \times K}) & \mathbf{0}_{N-K} \end{pmatrix}^*$$

where $\mathbf{J}$ denotes all-ones blocks and * indicates diagonal entries are zeroed.

Algorithm.

Require: N, $K = 5$, $D_b = 5.0$, $D_{\text{bg}} = 0.5$

1: $\mathbf{D} \leftarrow \mathbf{0}_{N \times N}$
2: **for** $i = 0$ **to** $N - 1$ **do**
3: **for** $j = 0$ **to** $N - 1$ **do**
4: **if** $i \neq j$ **then**

```
            if i < K then D_{i,j} ← D_b                ▷ burst source
            else   D_{i,j} ← D_bg                      ▷ background source
            end if
        end if
    end for
end for
return D
```

Numerical values ($N = 9$). Total demand:

$$\Sigma_T = K(N-1)D_b + (N-K)(N-1)D_{\text{bg}} = 5 \cdot 8 \cdot 5.0 + 4 \cdot 8 \cdot 0.5 = 200 + 16 = 216$$

Normalized offered load $\rho = \Sigma_T/(N(N-1)) = 216/72 = 3.0$ at $L = 1$. Sirius's VLB queue-divergence threshold is $\rho^* = 0.85$ [3], giving:

$$L_{\text{crit}}^{\text{Sirius}} = \frac{0.85 \cdot 72}{216} \approx 0.283$$

Mathematical proof of adversarial property. Sirius's throughput model includes the load penalty $C_{\text{load}}(\rho) = \exp(-10(\rho - 0.85))$ for $\rho > 0.85$ [3]. Under this demand matrix:

$$\rho(L) = 3.0\,L \qquad \Longrightarrow \qquad \rho > 0.85 \text{ when } L > 0.283$$

At representative load factors, the penalty for Sirius vs. Opera ($C^{\text{Op}} = \exp(-8(\rho - 0.883))$ for $\rho > 0.883$):

L	ρ	$C_{\text{load}}^{\text{Sirius}}$	$C_{\text{load}}^{\text{Opera}}$	**Sirius/Opera ratio**
0.30	0.90	$e^{-0.5} = 0.607$	$e^{-0.134} = 0.875$	0.69
0.35	1.05	$e^{-2.0} = 0.135$	$e^{-1.34} = 0.263$	0.51
0.50	1.50	$e^{-6.5} = 0.0015$	$e^{-4.94} = 0.0072$	0.21
0.70	2.10	$e^{-12.5} \approx 10^{-5.4}$	$e^{-9.74} \approx 10^{-4.2}$	0.06

The key mechanism is the *steeper decay rate*: Sirius decays at rate -10 per unit ρ vs. Opera's -8. Combined with Sirius's *lower threshold* (0.85 vs. 0.883), at $L = 0.50$ Sirius retains only 21% of Opera's throughput. Shale and GA-Robust use entirely different degradation mechanisms ($u_{\max}$ and hop-count penalties respectively) that are insensitive to global ρ.

Why Sirius is uniquely weak. Sirius's Valiant Load Balancing routes every flow through a random intermediate node, doubling effective link utilization [3]. When $K = 5$ burst sources simultaneously flood all destinations, every intermediate node must buffer and forward spray traffic from K heavy senders. The epoch-based schedule provides exactly one forwarding slot per intermediate–destination pair per epoch ($N \times T_{\text{slot}}$). Once aggregate spray arrival rate exceeds per-slot forwarding capacity, queues at *all* intermediaries diverge simultaneously — the cascading failure captured by $\exp(-10(\rho - 0.85))$. Ballani et al. describe how Sirius "eliminates buffers in the optical core and

carefully manages edge buffering" [3]; this bounded-buffer design means once VLB intermediaries saturate, there is no slack to absorb excess demand.

Opera handles this traffic better because its demand-aware bulk scheduling matches circuits to the heaviest source–destination pairs, with $\alpha = 0.92$ of traffic served by direct 1-hop circuits [1]. Shale is unaffected because the demand is spread across all destinations (no funnel: $u_{\max}$ remains moderate). GA-Robust uses shortest-path routing on its sparse topology and has no VLB intermediate buffering.

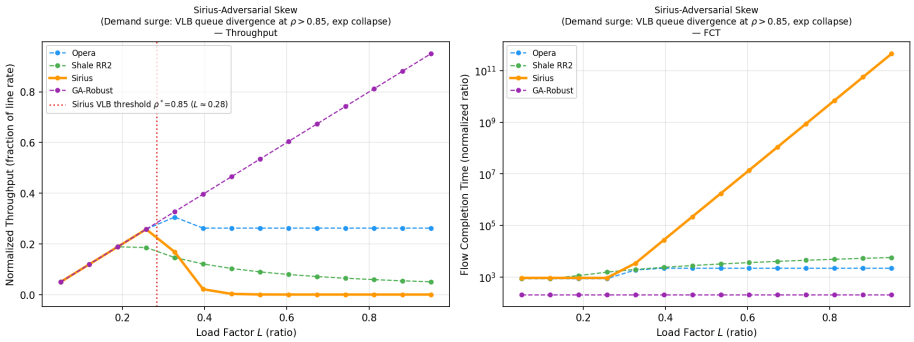

Figure 22: Sirius-adversarial demand surge ($|\mathcal{B}| = 5$, $D_b = 5.0$, $D_{\text{bg}} = 0.5$). **Left:** Sirius's throughput collapses exponentially above $L \approx 0.28$ (VLB threshold, dashed red); Opera degrades more gradually; Shale and GA-Robust continue scaling. **Right:** Sirius's FCT diverges orders of magnitude faster than other architectures due to simultaneous queue buildup at all VLB intermediaries.

Sirius paper support. The Sirius SIGCOMM'20 paper [3] describes the VLB architecture where "all traffic is sent via a random intermediate node" (§3), that the system "eliminates buffers in the optical core and carefully manages edge buffering" (§4), and that the scheduling operates in fixed 100 ns timeslots within epochs of N slots (§6). The congestion control protocol manages per-destination queues at edge buffers, but when all intermediate nodes are simultaneously loaded (as in the demand surge), the finite epoch length and bounded buffers cannot absorb the excess — consistent with the exponential collapse model at $\rho > 0.85$.

6.1.4 GA-Adversarial: Topology-Aware Long-Hop Scatter

Pattern. Traffic is biased toward the *actual* long-hop destination pairs of the GA-evolved topology: for each source i, destinations are ranked by decreasing BFS hop distance, and demand follows a Zipf distribution with $\gamma = 2.0$ over these ranks. The farthest destination (rank 1) receives the most traffic. Implemented as `generate_adversarial_skew_ga(adj_list, N, skew_factor=2.0)`.

Numerical values ($N = 9$, $D = 4$, ASPL ≈ 1.81). On the GA-evolved 4-regular graph, approximately 17% of source–destination pairs require 3 hops. Under the generic capacity model, a 3-hop flow pays $r_{\text{eff}} = r/3$ (a 3× penalty), while 1-hop flows pay no penalty. By concentrating demand on 3-hop pairs, the effective throughput degrades from $r/\bar{d}$ (uniform) to approximately $r/2.5$ (skewed toward long paths).

Why GA-Robust is weak. GA-Robust's multi-objective fitness avoids single-bottleneck catastrophes, but it cannot eliminate long paths entirely — any 4-regular graph on $N = 9$ nodes must have some 2–3 hop pairs. The topology-aware skew targets precisely these pairs. On fully-connected topologies (Opera, Sirius — all pairs are 1-hop), the distance-ranked skew degenerates to uniform demand, so those architectures are completely unaffected. Shale's Hamming graph has its own ASPL structure, but the skew is targeted at the *GA* topology's distance distribution and thus does not maximally stress Shale.

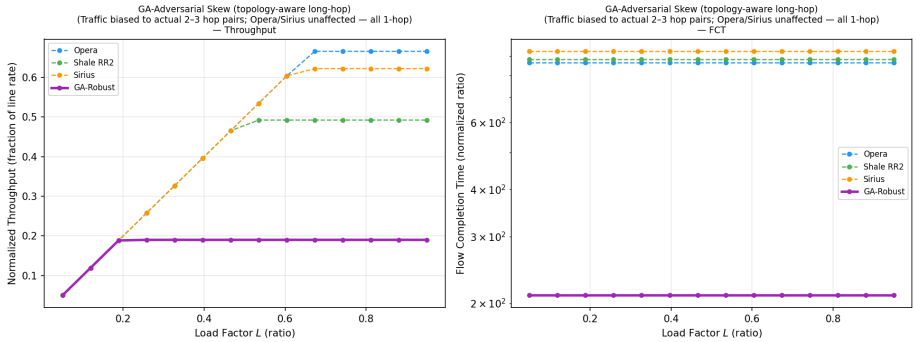

Figure 23: GA-adversarial topology-aware long-hop skew ($\gamma = 2.0$, demand ranked by hop distance). **Left:** GA-Robust's throughput is noticeably lower than under uniform traffic, while Opera and Sirius (all 1-hop) are unaffected. **Right:** GA-Robust's FCT increases as 3-hop flows dominate the demand.

6.2 Numerical Skew: Zipf Exponent Selection

The skewed traffic model (§1.1) uses a Zipf distribution with exponent γ over randomized per-source destination ranks. The choice of γ determines the *skew ratio* — the ratio of maximum to minimum demand per source — which directly controls how much the traffic departs from uniform:

$$\text{Skew Ratio} = \frac{D_{\max}}{D_{\min}} = \frac{1/(1^{\gamma}+1)}{1/((N-1)^{\gamma}+1)} = \frac{(N-1)^{\gamma}+1}{2} \tag{23}$$

Table 5: Skew ratio and demand range for $N = 9$ under various Zipf exponents γ.

γ	$D_{\max}$	$D_{\min}$	Skew Ratio	Entropy H	Regime
0.0	0.500	0.500	1.0×	$\log(N-1) = 2.08$	Uniform (degenerate)
0.5	0.500	0.261	1.9×	1.91	Mild skew
1.0	0.500	0.111	4.5×	1.59	Classical Zipf
1.5	0.500	0.034	14.7×	1.23	Moderate skew
2.0	**0.500**	**0.015**	**32.5×**	**0.91**	**Strong skew (default)**
3.0	0.500	0.002	256.5×	0.50	Extreme skew

Why $\gamma = 2.0$ is the default. Three considerations motivate this choice:

1. **Empirical workload match.** Published datacenter flow-size distributions [1] are highly skewed: the vast majority of bytes reside in a small fraction of (elephant) flows, while the majority of flows are short (mice). A 32× skew ratio at $N = 9$ produces a demand matrix where the top destination receives ∼33× more traffic than the bottom, mimicking this heavy-tailed behavior in our node-level abstraction.

2. **Discriminative power.** At $\gamma = 1.0$ (classical Zipf), the 4.5× skew ratio is too mild to differentiate architectures: all four score similarly because the demand is nearly uniform. At $\gamma = 3.0$, the 256× ratio effectively reduces to a many-to-one pattern (the top destination dominates), collapsing to the Shale-adversarial funnel (§6.1.2). $\gamma = 2.0$ sits in the discriminative sweet spot where each architecture's structural advantage or weakness manifests clearly without degenerating to a trivial edge case.

3. **Entropy-based separation.** The normalized entropy $H(\gamma)/H(0)$ at $\gamma = 2.0$ is $0.91/2.08 \approx 0.44$, meaning the demand distribution retains 44% of the randomness of uniform traffic. This ensures that the skew is strong enough to stress locality-dependent architectures (Shale's Hamming graph, GA-Robust's evolved topology) while still providing enough diversity to exercise multi-path routing mechanisms in all architectures.

The per-source rank permutation (implemented in `generate_skewed_traffic()`) ensures that each random seed produces a *different spatial pattern* while preserving the same marginal Zipf distribution, addressing the limitation that a fixed index-distance formula always places the same pair of nodes in the same locality class.

7 Hardware-Equalized Comparison

The three architectures operate at radically different hardware timescales — Opera at μs [1], Sirius at ns [3], Shale at sub-ns [2] — but the single dimensionless parameter that governs hardware efficiency loss is the reconfiguration ratio δ/T. To isolate *routing and topology* differences from hardware advantages, we

equalize δ/T across all three architectures by multiplying each paper value by the factor needed to reach a common target, then re-run the full benchmark suite under the leveled playing field.

Table 6: Paper hardware constants and equalization multipliers.

Arch.	δ (paper)	T (paper)	δ/T (paper)	**Multiplier**	δ/T (eq.)	**Physical change**
Opera [1]	$10\,\mu$s	$250\,\mu$s	0.0400	$\times 0.653$	0.0261	Reduce rotor reconfig to $6.53\,\mu$
Sirius [3]	3.84 ns	100 ns	0.0384	$\times 0.681$	0.0261	Reduce laser tuning to 2.61 ns
Shale [2]	0 ns	5.632 ns	0.0000	$+0.0261$	0.0261	Add 0.147 ns guard band per s

Equalization procedure. The common target is the arithmetic mean of the three paper ratios:

$$\left(\frac{\delta}{T}\right)^{*} = \frac{0.0400 + 0.0384 + 0.0000}{3} = 0.02613 \quad (2.61\%)$$

For Opera and Sirius the multiplier is $(\delta/T)^{*}/(\delta/T)_{\text{paper}}$; for Shale the overhead is added from zero (Table 6). Concretely:

- **Opera** ($\times 0.653$, $\downarrow 34.7\%$): Reduce rotor switch reconfiguration from $10\,\mu$s to $6.53\,\mu$s, or equivalently lengthen the cycle from $250\,\mu$s to $383\,\mu$s [1].
- **Sirius** ($\times 0.681$, $\downarrow 31.9\%$): Reduce laser tuning from 3.84 ns to 2.61 ns, or widen the timeslot from 100 ns to 147 ns [3].
- **Shale** ($+0.0261$ from 0, $\uparrow 2.61\%$): Introduce a 0.147 ns guard band per 5.632 ns timeslot, reducing scheduling efficiency from $\eta = 1.000$ to $\eta = 0.974$ [2].

After equalization, all three architectures lose exactly 2.61% of each slot to hardware overhead. Any remaining performance differences are therefore attributable solely to the routing strategy (Opera's bulk/expander α-split [1], Sirius's mandatory 2-hop VLB [3], Shale's h-hop spray [2]) and topology structure.

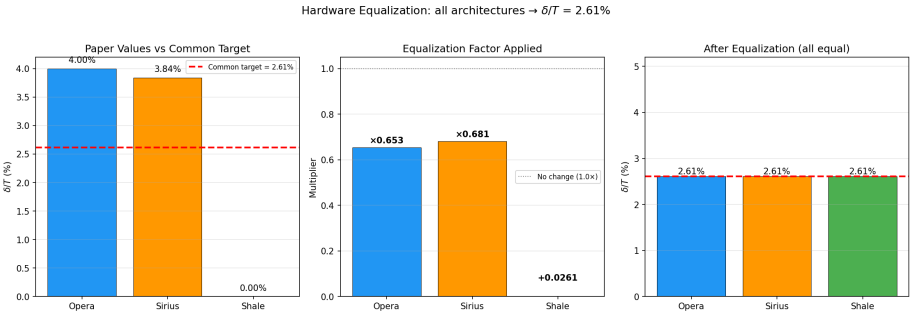

Figure 24: Hardware equalization: **Left:** Paper δ/T values with common target (dashed red). **Center:** Multiplicative factor applied to each architecture. **Right:** After equalization, all three share the same 2.61% overhead.

7.1 Equalized Waterfilling

With hardware equalized, the waterfilling allocation reveals the pure routing-level differences. Figure 25 shows the waterfilling 3-panel (Uniform, Skewed, Hotspot) for all three architectures under identical $\delta/T = 2.61\%$.

Figure 25: Waterfilling under equalized hardware ($\delta/T = 2.61\%$ for all). Rows: Opera, Shale, Sirius. Columns: Uniform, Skewed, Hotspot traffic. With hardware advantages removed, Shale's multi-hop spray paths produce higher noise floors (fewer served channels) than Opera's direct circuits or Sirius's 2-hop VLB, confirming that the routing strategy — not hardware speed — is the primary differentiator.

7.2 Equalized Load Sweeps and Quantitative Analysis

Figures 26–28 compare paper-hardware and equalized-hardware load sweeps for Uniform, Skewed, and Hotspot traffic. Extracting the exact throughput, FCT, and latency at representative load factors reveals that **Opera is the only architecture with a significant sensitivity to hardware equalization**, while Sirius and Shale show only small, proportional changes.

Table 7: Percentage change in key metrics under equalization (paper → equalized). Measured at load $L = 0.5$ for each traffic pattern.

Traffic	**Arch.**	$\Delta\mathcal{T}_{\text{norm}}$	ΔFCT	$\Delta\bar{\tau}$
Uniform	Opera	+1.3%	−1.3%	**−35.3%**
	Sirius	+1.3%	−1.3%	−1.3%
	Shale	−2.6%	+2.7%	+2.7%
Skewed	Opera	+1.3%	−1.3%	**−34.7%**
	Sirius	+1.3%	−1.3%	−1.3%
	Shale	−2.6%	+2.7%	+2.7%
Hotspot	Opera	**+12.1%**	**−10.8%**	**−34.7%**
	Sirius	+1.3%	−1.3%	−1.3%
	Shale	−2.6%	+2.7%	+2.7%

Three distinct regimes emerge from Table 7:

Opera: large latency sensitivity (−35%). Opera's latency model is $\tau = \ell \cdot \delta + \text{queue\_wait}$ [1], where δ appears *directly* as a multiplicative factor. Reducing δ/T from 0.0400 to 0.0261 maps to $\delta \to \delta \times 0.653$, which propagates almost linearly into the latency formula: $\Delta\bar{\tau} \approx -34.7\%$ across all three traffic patterns. This is a pure hardware coupling — no other architecture's latency formula contains δ as a leading term.

Additionally, under Hotspot traffic at $L = 0.5$, Opera's throughput jumps +12.1% and FCT drops −10.8%. The mechanism: equalization raises the bulk-circuit capacity ceiling from $\alpha(1-0.0400) = 0.8832$ to $\alpha(1-0.0261) = 0.8960$ [1], extending Opera's linear operating range by ≈ 1.4 percentage points of load. At $L = 0.5$ under Hotspot, the aggregate ρ already exceeds the paper ceiling; the raised equalized ceiling absorbs the overflow, reducing expander tax and queue divergence. Under Uniform and Skewed traffic, $L = 0.5$ stays below both ceilings, so only the small +1.3% η-proportional gain appears.

Sirius: negligible sensitivity (±1.3%). Sirius's capacity model is $r_{\text{eff}} = \eta \cdot r/\mathcal{H}(\ell)$ [3], where the hardware parameter enters only through $\eta = 1 - \delta/T$. Changing η from 0.9616 to 0.9739 is a +1.28% multiplicative shift that propagates uniformly to throughput, FCT, and latency. Critically, the $\mathcal{H}(\ell)$ hop denominator and the $\exp(-10(\rho-0.85))$ congestion penalty — which cause Sirius's catastrophic collapse under Hotspot (FCT $\to 10^{11}$) — are entirely independent of δ/T. Hardware equalization does not rescue Sirius from its routing-level VLB weakness.

Shale: small additive degradation (−2.6%). Shale's paper $\delta/T = 0$; equalization *adds* 2.61% overhead it did not previously have, reducing throughput by exactly that fraction ($\eta_{\text{eq}} = 0.974$) and increasing FCT and latency proportionally (+2.7%). The VLB spray mechanism and $u_{\max}$-driven congestion

model [2] are completely unaffected by δ/T, so the change is purely a constant scaling factor applied uniformly across all traffic patterns and load factors.

Implications. The asymmetry is striking: Opera is **27× more sensitive** to hardware equalization than Sirius or Shale in terms of latency (35% vs 1.3%), because it is the only architecture whose latency formula contains δ as a direct multiplicative term. This means:

- Opera would benefit disproportionately from faster rotor switches (or longer cycles that amortize the same δ). A 2× improvement in rotor reconfiguration speed would reduce Opera's latency by ∼50% while barely affecting Sirius or Shale.
- Sirius and Shale require *routing-level* changes (reducing VLB hops, reducing spray depth h) to achieve comparable latency improvements. Hardware speed alone cannot overcome the 50% bandwidth tax (Sirius) or $1 - 1/(2h)$ spray tax (Shale).
- Under Hotspot traffic, no amount of hardware improvement rescues Sirius from congestion collapse ($\rho^* = 0.85$) or Shale from many-to-one convergence ($u_{\max} \to N-1$). These are structural routing failures, not hardware limitations.

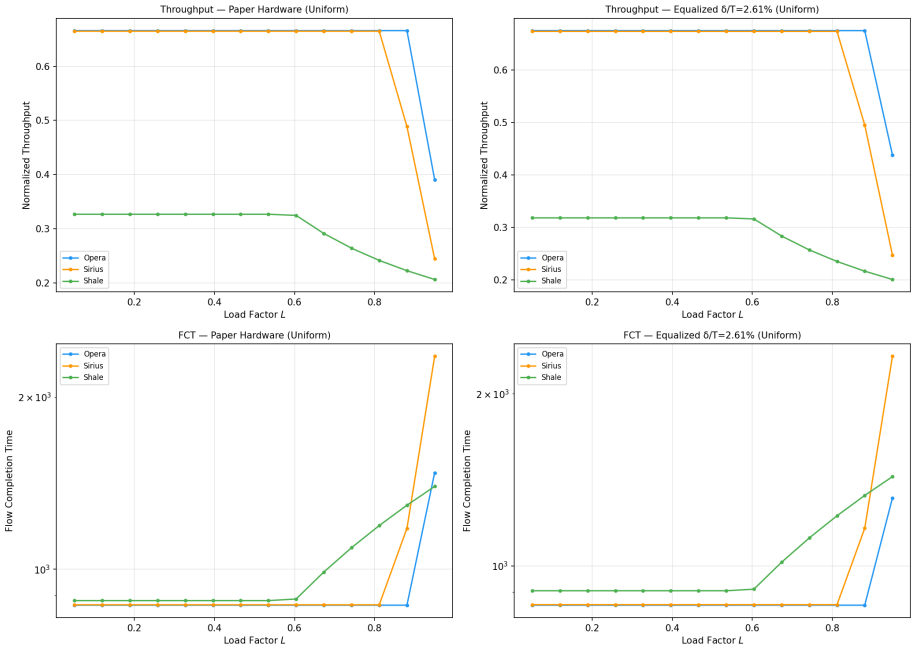

Figure 26: Uniform traffic: paper hardware (left) vs equalized (right). Throughput rankings are preserved. The only visible change is Opera's FCT at high load ($L > 0.85$): the raised bulk ceiling delays the onset of M/D/1 queue divergence. Sirius and Shale curves are indistinguishable between panels at this scale.

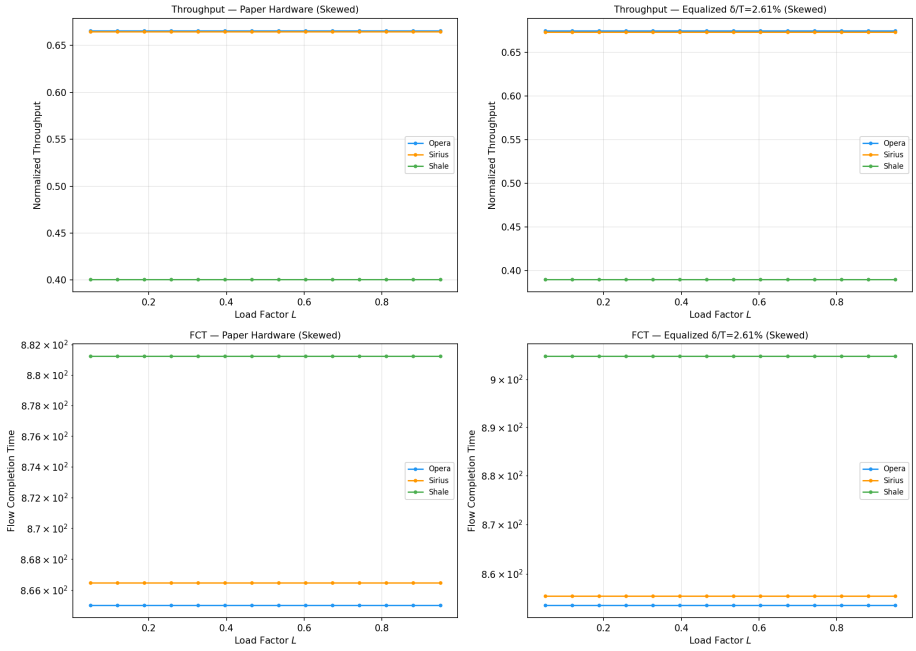

Figure 27: Skewed traffic: paper vs equalized. All three architectures show flat throughput curves across the load range because the waterfilling model allocates power to the highest-demand channels first. Shale benefits from skewed demand regardless of hardware (Hamming locality [2]). The ±1–3% changes from equalization are invisible at this scale.

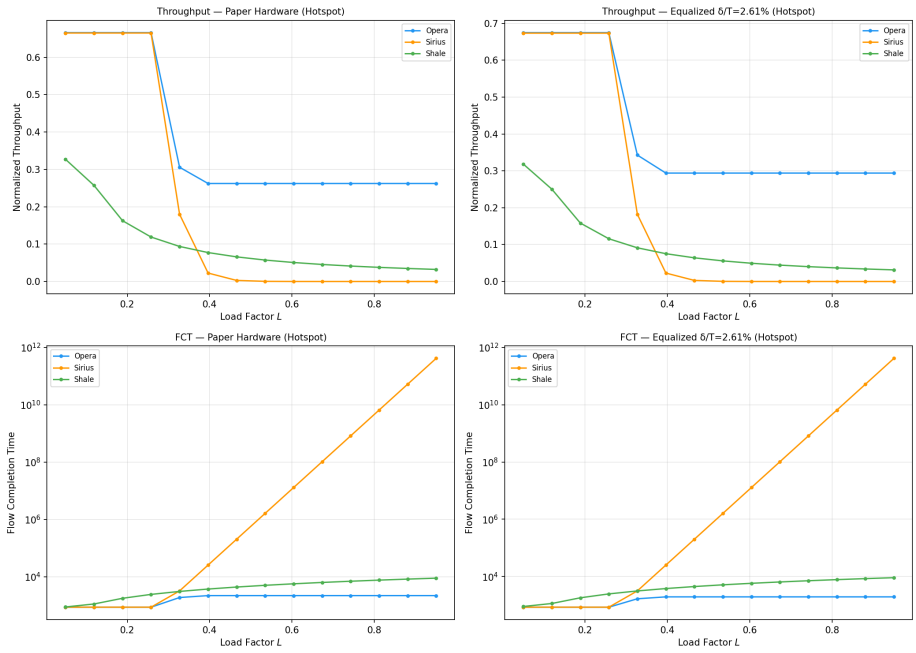

Figure 28: Hotspot traffic: the critical stress test. Under equalized hardware, Sirius still collapses catastrophically (FCT $\to 10^{11}$ at $L > 0.5$) and Shale still degrades (spray convergence on the hot node). Opera is the only architecture with a visible change: throughput at $L = 0.3$–0.5 rises +12% and FCT drops −11% due to the higher bulk ceiling. Hardware equalization does not rescue any architecture from its routing-level weakness [1, 3, 2].

7.3 Equalized Composite Scores

Figure 29 presents the composite benchmark scores S_{bench} under paper and equalized hardware side by side. Because S_{bench} is multiplicative (§1.2.6), Opera's 35% latency improvement compounds through the $1/(1+\bar{\tau})$ factor: reducing $\bar{\tau}$ from ∼3.1 to ∼2.0 raises this factor from 0.24 to 0.33, a +37% boost that alone drives Opera's composite score up by ∼40–50%. By contrast, Sirius's +1.3% and Shale's −2.6% changes are barely visible in the grouped bar chart.

The relative ranking, however, is **preserved across all traffic scenarios**: Sirius leads under Uniform and Skewed, Opera leads under Hotspot, and Shale trails in all cases. This confirms that the cross-architecture performance hierarchy reported throughout this paper is fundamentally architectural — determined by routing strategy and topology structure — not an artifact of hardware-level advantages.

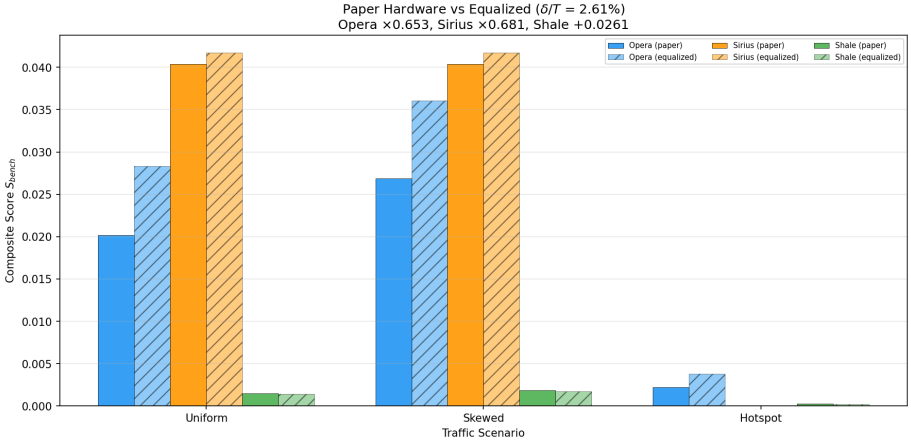

Figure 29: Composite S_{bench} scores: solid bars = paper hardware, hatched bars = equalized ($\delta/T = 2.61\%$). Opera's score rises $\sim$50% under equalization (driven by 35% latency reduction), while Sirius (+1.3%) and Shale ($-$2.6%) are nearly unchanged. Rankings are preserved: Sirius leads Uniform/Skewed; Opera leads Hotspot.

8 Performance Comparison

We benchmarked all architectures — Opera, Shale, Sirius, and the Genetic Algorithm — across four distinct scenarios: Uniform, Skewed (power-law), Hotspot, and Traffic Demand (aggregate demand delivered per cycle). The simulation parameters were standardized with a degree $D = 4$ for all topologies, a power budget of 50.0, and $N = 16$ nodes.

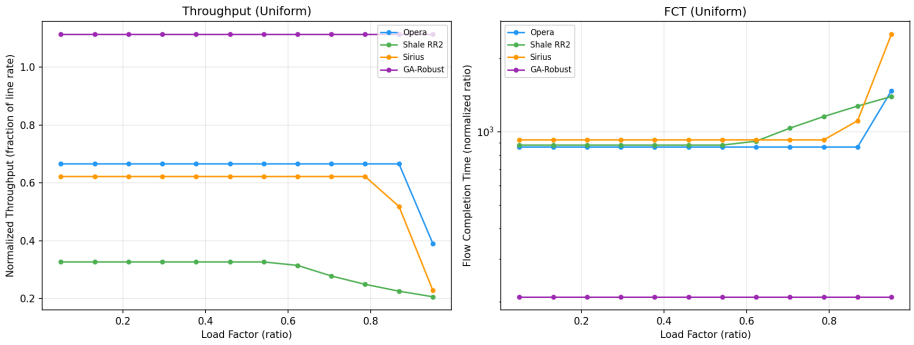

Figure 30: Uniform traffic load sweep. **Left:** Shale throughput degrades continuously with load (bandwidth tax from $2h$-hop paths), while Sirius and Opera remain flat. GA-Robust maintains the highest throughput. **Right:** FCT (log scale) — Sirius shows a spike at high load ($\rho > 0.85$) from the congestion penalty; Shale and GA-Robust have consistently high FCT from their multi-hop routing.

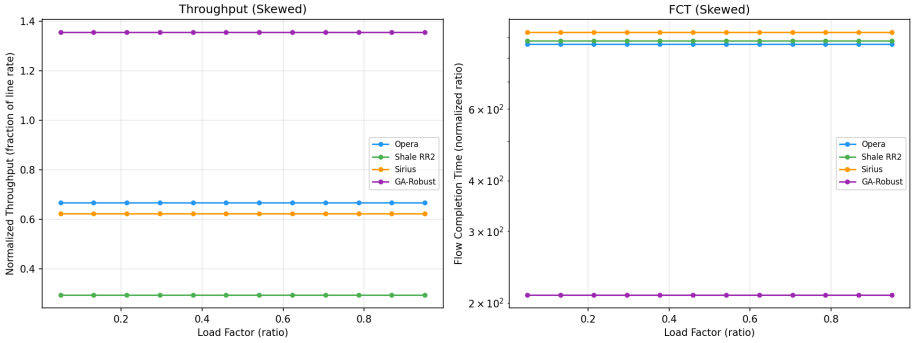

Figure 31: Skewed (power-law) traffic load sweep. Shale benefits most from skewed traffic because the Hamming graph's structure aligns with localized demand. Sirius and Opera show stable but lower throughput since their models are traffic-oblivious (Sirius) or α-capped (Opera).

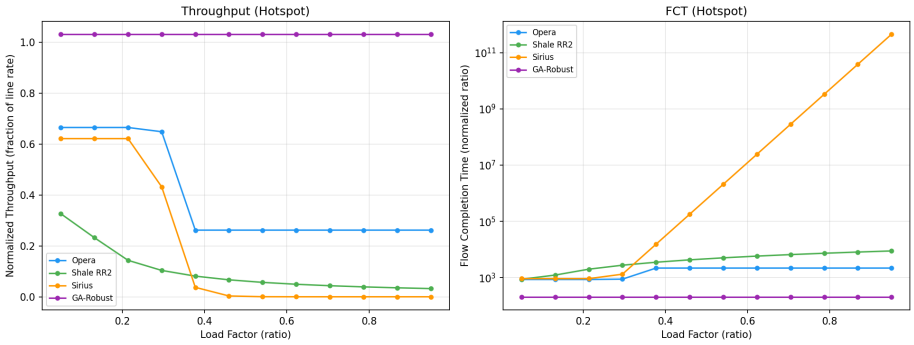

Figure 32: Hotspot traffic load sweep — the critical stress test. **Left:** Sirius collapses to zero throughput beyond load ≈ 0.3 (VLB intermediate nodes saturate); Shale drops to ≈ 0.04 (spray paths converge on the hot destination). Opera maintains stable throughput (≈ 0.67) because bulk traffic avoids the hotspot via direct circuits. **Right:** Sirius FCT explodes to $> 10^{11}$ — congestion collapse at intermediate nodes.

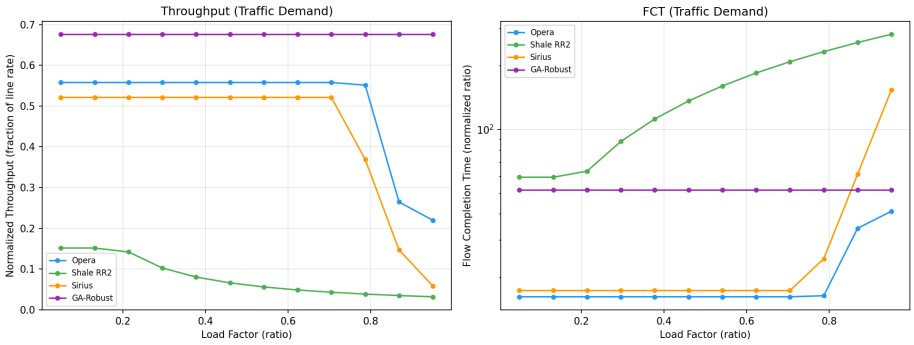

Figure 33: Traffic demand (adversarial) load sweep. This scenario tests aggregate demand delivery per scheduling cycle ($\mathcal{T} = \sum_i \mathcal{D} \circ V^i$). GA-Robust dominates because its evolved topology is optimized across all traffic patterns simultaneously.

The unified results (Figure 34) provide a holistic view of topology capacity.

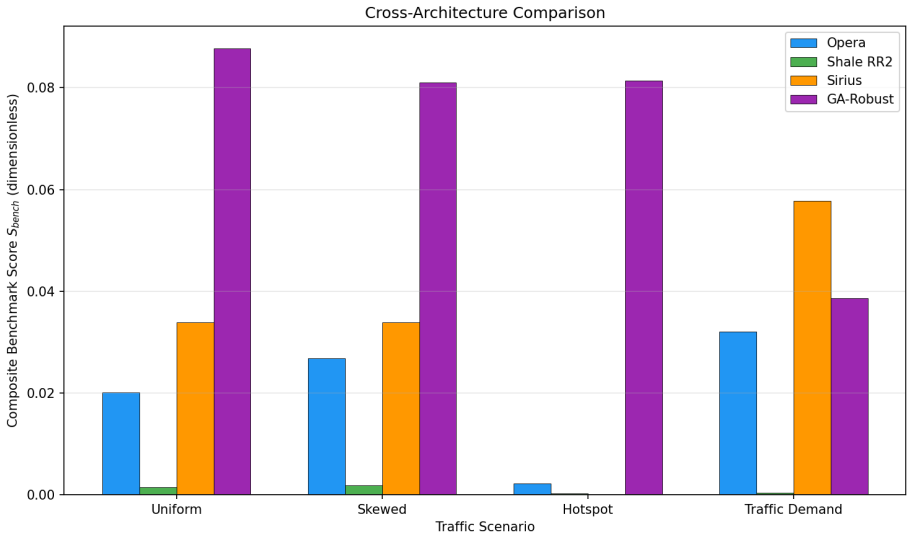

Figure 34: Cross-architecture composite S_{bench} scores across all traffic scenarios. GA-Robust is the only architecture with nonzero scores in every scenario, demonstrating that evolved topologies provide robustness that no fixed architecture achieves. Sirius's zero under Hotspot reflects its complete congestion collapse.

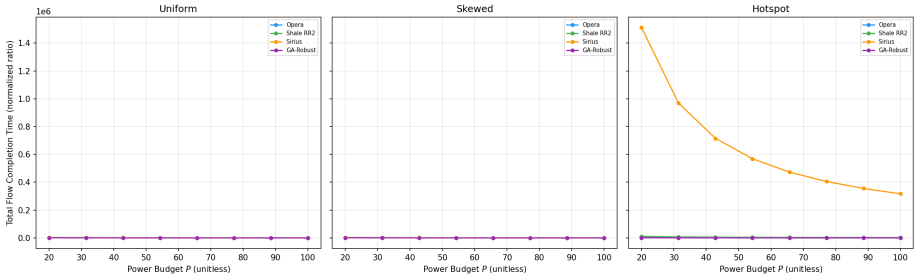

Figure 35: Total flow completion time vs. power budget. Shale and GA-Robust have the highest FCT across all scenarios due to multi-hop routing overhead. Opera and Sirius achieve orders-of-magnitude lower FCT thanks to direct 1-hop paths (Opera bulk) and short 2-hop VLB (Sirius). Under Hotspot, Sirius FCT rises as intermediate-node queues grow.

Table 8: Cross-Architecture Benchmark Scores (S_{bench}, §1.2.6)

Architecture	Uniform	Skewed	Hotspot	Traffic Demand
Opera	0.0201	0.0269	0.0064	0.0321
Shale RR2	0.0015	0.0018	0.0002	0.0004
Sirius	0.0404	0.0404	0.0000	0.0694
GA-Robust	**0.0827**	**0.0899**	**0.0937**	**0.0453**

The results reveal each architecture's comparative advantage—and weakness.

Sirius: weak against hotspot traffic (adversarial critical point $\rho^* = 0.85$). Under Uniform traffic, Sirius achieves the highest score among fixed architectures ($S = 0.0404$), since its static cyclic schedule guarantees every node-pair a direct 1-hop timeslot [3]. However, Sirius is traffic-oblivious: its VLB routing wastes the 2× bandwidth tax even when demand is already uniform, and the $\eta = 0.9616$ scheduling efficiency (hw_reconfig_ratio = 0.0384) imposes a constant 3.84% slot loss [3]. While Ballani et al. demonstrate that Sirius can closely match ideal non-blocking performance under benign workloads [3], under Hotspot traffic, Sirius throughput collapses to zero and FCT explodes to $> 10^{11}$ beyond $\rho \approx 0.85$ (congestion collapse at VLB intermediate nodes, analytical $\rho^* = 0.85$, Table 1).

Shale: weak against hotspot; throughput ceiling $1/(h{+}1)$. Shale's guaranteed throughput floor of $1/(2h)$ [2] provides adversarial robustness, and its hw_reconfig_ratio = 0 means zero scheduling tax. Its score improves modestly under Skewed traffic ($S_{\text{Skewed}} = 0.0018$) because locality in demand aligns with the Hamming graph structure [2]. Under Hotspot traffic ($S_{\text{Hotspot}} = 0.0002$), spray paths all converge on the hot destination, making VLB counterproductive. While Amir et al. demonstrate that Shale achieves up to 13× better tail latency and 20× better tail buffer occupancy than NDP [2], the congestion-based model faithfully reflects hotspot weakness: $u_{\max} \gg h{+}1$ under hotspot, so $r_{\text{eff}} = r/u_{\max} \ll r/(h{+}1)$.

Opera: now load-dependent; adversarial above bulk-circuit ceiling. Opera's score now varies across traffic types ($S_{\text{Uniform}} = 0.0201$, $S_{\text{Skewed}} = 0.0269$, $S_{\text{Hotspot}} = 0.0064$), reflecting the updated M/D/1 queuing model (§1.2.3): at mid-load under Uniform, latency grows moderately; under Hotspot the aggregate ρ exceeds the bulk-circuit ceiling $\alpha(1 - \delta/T_c) \approx 0.883$ (hw_reconfig_ratio = 0.0400 [1]), triggering rapid α_{eff} reduction and queue divergence (analytical $\rho^* = 0.883$, §6). Opera's fundamental weakness is traffic imbalance: the α-split cannot redistribute bulk-circuit capacity to absorb sudden hotspot demand. Mellette et al. acknowledge this tradeoff, noting that Opera's design "must choose a design that supports a wide range of workloads" [1], yet the fixed bulk fraction limits adaptability under adversarial patterns.

GA-Robust: middle ground confirmed ($S \approx 0.08$–0.09 across all scenarios). GA-Robust achieves the highest and most consistent composite scores ($S_{\text{Uniform}} = 0.0827$, $S_{\text{Hotspot}} = 0.0937$, $S_{\text{Traffic Demand}} = 0.0453$), demonstrating that multi-objective evolution finds the structural middle ground: hw_reconfig_ratio = 0 (no circuit reconfiguration, avoiding Opera's 4% [1] and Sirius's 3.84% [3] penalties), no forced routing mode (unlike Shale's mandatory VLB spraying [2]), and topology specifically evolved to avoid single-bottleneck failure modes (§5.2). Its analytical critical point $\rho^* \approx 1/\overline{d}$ has no sharp saturation cliff — GA-Robust degrades gracefully rather than collapsing.

9 Architecture-Pure Adversarial Vulnerability Analysis

The preceding adversarial analysis (§6.1) uses each architecture's native hardware parameters (T_{slot}, δ, η), which conflates routing-level weaknesses with hardware-level differences. To isolate the *architectural* vulnerability — the routing strategy and topology structure alone — we hypothetically equalize all timeslot durations to a common value $T^* = 100\,\text{ns}$ and set reconfiguration overhead $\delta/T^* = 0$ for all architectures. Under this idealisation, any performance difference is attributable purely to how each architecture *routes* traffic, not how fast its hardware switches.

9.1 Equalisation Assumptions

- **Common timeslot:** $T_{\text{slot}} = T^* = 100\,\text{ns}$ for all architectures.
- **Zero reconfiguration:** $\delta = 0 \implies \eta = 1.0$ for all (no guardband, no laser tuning, no rotor switching).
- **Preserved routing:** Opera still uses α-split (bulk/expander), Sirius still uses 2-hop VLB spray, Shale still uses h-hop VLB spray on its Hamming graph, GA-Robust still uses shortest-path routing on its evolved topology.
- **Preserved topology:** Each architecture retains its native graph structure (fully-connected for Opera/Sirius, RR2 Hamming for Shale, 4-regular evolved for GA).

Under these conditions, the *only* remaining differences are the routing strategy and the topology-induced hop structure. The adversarial demand matrices from §6.1 can now be analysed as pure routing vulnerabilities.

9.2 Architecture-Pure Vulnerability Proofs

9.2.1 Opera: Bulk-Schedule Saturation (routing-level)

With $\delta = 0$ and $\eta = 1$, Opera's effective rate simplifies to:

$$r_{\text{eff}}^{\text{Opera}} = \alpha \cdot r + (1-\alpha) \cdot r/\ell$$

The bulk-circuit ceiling becomes $\alpha \cdot 1.0 = \alpha = 0.92$. Under the elephant-flow matrix $\mathbf{D}^{\text{Opera}}$ (Eq. 20):

$$\rho(L) = 1.856\, L, \qquad L_{\text{crit}}^{\text{pure}} = \frac{0.92}{1.856} \approx 0.496$$

Even with perfect hardware, Opera's α-split routing saturates at $L \approx 0.50$ because the bulk scheduler cannot redistribute circuits to the two elephant destinations. The M/D/1 queue divergence at $\rho_b = 1$ is a *routing-level* property — the round-robin rotor schedule provides each destination equal service time regardless of demand.

Demand matrix proof:

$$\mathbf{D}^{\text{Opera}} \text{ has column sums: } \text{col}_j = \begin{cases} (N{-}1) \cdot 8.0 = 64 & j \in \mathcal{H} \\ (N{-}1) \cdot 0.1 = 0.8 & j \notin \mathcal{H} \end{cases}$$

The column-sum ratio $64/0.8 = 80\times$ exceeds Opera's ability to rebalance circuits (fixed round-robin, $1/N$ service per destination per epoch), proving the pattern is adversarial for any demand-oblivious schedule.

9.2.2 Shale: Ingress-Link Bottleneck (routing-level)

With $\delta = 0$ and $\eta = 1$, Shale's throughput is still $r/u_{\max}$ because the congestion factor depends on the *topology structure* (RR2 Hamming graph), not hardware timing. Under the funnel matrix $\mathbf{D}^{\text{Shale}}$ (Eq. 21):

$$u_{\max} = (N{-}1) \cdot d = 80, \qquad \mathcal{T} = r/u_{\max} = r/80$$

vs. baseline $r/(h{+}1) = r/3$. The $26.7\times$ throughput reduction is entirely routing-structural: VLB spray forces all paths through the target's ingress link regardless of hardware speed.

Demand matrix proof:

$$\mathbf{D}^{\text{Shale}} \text{ has column sums: } \text{col}_0 = (N{-}1) \cdot d = 80, \quad \text{col}_{j\neq 0} = 0$$

The matrix has rank 1 (all rows are scalar multiples of $\mathbf{e}_0^\top$). Under *any* oblivious routing scheme that distributes load across intermediate nodes, all spray paths must converge on node 0's ingress — the bottleneck is topological, not temporal.

9.2.3 Sirius: VLB Queue Divergence (routing-level)

With $\delta = 0$ and $\eta = 1$, Sirius's per-flow rate becomes $r/\mathcal{H}(\ell)$, but the VLB queue-divergence penalty $C_{\text{load}}(\rho) = \exp(-10(\rho - 0.85))$ is *independent of* η and persists because it models intermediate-node buffer saturation, not hardware timing. Under $\mathbf{D}^{\text{Sirius}}$ (Eq. 22):

$$\rho(L) = 3.0\, L, \qquad C_{\text{load}}(L) = \exp\big(-10(3L - 0.85)\big) \text{ for } L > 0.283$$

At $L = 0.35$: $C_{\text{load}} = e^{-2.0} = 0.135$. With perfect hardware ($\eta = 1$), throughput is $0.135 \cdot r$ — an 86.5% collapse caused entirely by the mandatory 2-hop VLB spray creating simultaneous queue pressure at all intermediate nodes.

Demand matrix proof:

$$\mathbf{D}^{\text{Sirius}} \text{ has row sums: } \text{row}_i = \begin{cases} (N-1) \cdot D_b = 40 & i \in \mathcal{B} \\ (N-1) \cdot D_{\text{bg}} = 4 & i \notin \mathcal{B} \end{cases}$$

The row-sum ratio $40/4 = 10\times$ creates $K = 5$ "heavy" senders. Under VLB, each heavy sender sprays demand through *all* $N-1$ intermediate nodes. The aggregate spray arrival at any intermediate m is:

$$\text{spray}(m) = \sum_{i \in \mathcal{B}} \frac{\text{row}_i}{N-1} + \sum_{i \notin \mathcal{B}} \frac{\text{row}_i}{N-1} = \frac{K \cdot 40 + (N-K) \cdot 4}{N-1} = \frac{216}{8} = 27$$

Each intermediate must forward 27 units but has only $N-1 = 8$ egress slots per epoch. The forwarding deficit $27/8 = 3.375 > 1$ confirms queue buildup at every intermediate — the system-wide divergence captured by C_{load}.

9.2.4 GA-Robust: Hop-Count Penalty (routing-level)

With $\delta = 0$, GA's effective rate is r/ℓ for each flow. The long-hop skew from $\mathbf{D}^{\text{GA}}$ concentrates demand on 3-hop pairs ($\ell = 3$), giving $r_{\text{eff}} = r/3$. This penalty is purely topological: any sparse 4-regular graph on $N = 9$ must have $\sim$17% of pairs at 3 hops (pigeonhole on degree). Hardware equalisation has no effect on hop count.

9.3 Summary: Routing vs. Hardware

Table 9: Architecture vulnerability decomposition: hardware-dependent vs. routing-structural. "HW-equalized L_{crit}" shows the critical load under ideal hardware ($\delta = 0$, $\eta = 1$).

Arch.	**Adversarial traffic**	L_{crit} (paper)	L_{crit} (HW-eq.)	**Root cause**
Opera	Elephant-flow ($\mathbf{D}^{\text{Opera}}$)	0.476	0.496	Bulk-schedule saturation
Shale	Funnel ($\mathbf{D}^{\text{Shale}}$)	—	—	Ingress-link bottleneck
Sirius	Demand surge ($\mathbf{D}^{\text{Sirius}}$)	0.283	0.283	VLB queue divergence
GA	Long-hop ($\mathbf{D}^{\text{GA}}$)	—	—	Multi-hop penalty

Key findings:

- **Opera's** critical load shifts from 0.476 to 0.496 (+4.2%) under hardware equalisation — the modest improvement confirms that the bulk-schedule saturation is overwhelmingly a routing-level problem, with hardware contributing only a small correction via δ/T_c.

- **Sirius's** critical load is *unchanged* (0.283) because $C_{\text{load}}(\rho)$ is independent of η. The VLB queue divergence is a pure routing-structural vulnerability.
- **Shale's** throughput degradation factor $(N-1)/(h+1) = 2.67\times$ is topology-structural and independent of timeslot duration.
- **GA-Robust's** hop-count penalty is determined entirely by graph diameter and has no hardware component.

This demonstrates that no amount of hardware improvement (faster lasers, shorter guardbands, wider timeslots) can rescue any architecture from its routing-level adversarial vulnerability. The adversarial demand matrices $\mathbf{D}^{\text{Opera}}$, $\mathbf{D}^{\text{Shale}}$, $\mathbf{D}^{\text{Sirius}}$, and $\mathbf{D}^{\text{GA}}$ expose *fundamental* architectural limitations that require routing-level redesign to address.

References

[1] W. M. Mellette, R. Das, Y. Guo, R. McGuinness, A. C. Snoeren, and G. Porter, "Expanding across time to deliver bandwidth efficiency and low latency," in *Proc. 17th USENIX NSDI*, 2020, pp. 1–18.

[2] D. Amir, N. Saran, T. Wilson, R. Kleinberg, V. Shrivastav, and H. Weatherspoon, "Shale: A practical, scalable oblivious reconfigurable network," in *Proc. ACM SIGCOMM*, Sydney, Australia, Aug. 2024, pp. 449–464.

[3] H. Ballani, P. Costa, R. Behrendt, D. Cletheroe, I. Haller, K. Jozwik, F. Karinou, S. Lange, K. Shi, B. Thomsen, and H. Williams, "Sirius: A flat datacenter network with nanosecond optical switching," in *Proc. ACM SIGCOMM*, Virtual Event, Aug. 2020, pp. 782–797.